

目 录

1 课题概述	1
1.1 选题背景	1
1.2 课题内容与方法	2
1.2.1 研究方向	2
1.2.2 要解决的问题及所需信息	3
1.2.3 解决问题的方法	4
1.3 课题目标和使用技术	5
2 数据采集	7
2.1 数据来源	7
2.2 数据采集方案	7
2.3 采集方案实施	8
2.3.1 菜单任务构建实施过程	8
2.3.2 主抓取程序实施过程	16
3 数据预处理	28
3.1 数据预处理目标	28
3.2 原始数据读取	28
3.3 删除无关列与冗余字段	30
3.4 关键特征变量构造	30
3.4.1 “急招”标志变量构造	31
3.4.2 岗位标签结构化处理	32
3.4.3 薪资字段数值化处理	32
3.4.4 实习时长与每周工作天数字段提取	33
3.5 预处理结果写入数据库	34
4 数据存储	36
4.1 选择数据存储方式	36
4.2 构建数据存储结构	36
4.3 展示数据存储结果	38
5 数据提取	42
5.1 问题一：不同岗位方向的数量分布如何	42
5.2 问题二：互联网 IT 类实习岗位主要集中在哪些城市	43
5.3 问题三：高薪岗位通常具有什么特征	44
5.4 问题四：企业对技术技能的要求主要体现在哪些关键词上	45
5.5 问题五：岗位平均日薪分布情况	47

6 数据可视化与分析	50
6.1 岗位方向数量分布可视化分析	50
6.2 城市岗位数量分布可视化分析	51
6.3 技能关键词需求可视化分析	52
6.4 急招岗位数量与占比可视化分析	54
6.5 岗位标签词云可视化分析	55
7 总结	58
8 参考文献	59

1 课题概述

1.1 选题背景

随着数字经济、互联网平台经济和人工智能技术的持续发展，数字产业正在成为推动经济结构转型和就业岗位变化的重要力量。根据《数字中国发展报告（2024年）》，2024年我国数字经济核心产业规模稳步扩大，占国内生产总值比重达10%左右，数据生产量达到41.06ZB，同比增长25%，生成式人工智能用户约2.5亿，说明数字技术已经深度嵌入产业发展和社会生活之中^[1]。与此同时，人力资源社会保障部等部门发布的《加快数字人才培养支撑数字经济发展行动方案（2024—2026年）》也提出，要围绕大数据、人工智能、智能制造、集成电路、数据安全等数字领域新职业，加快数字技术人才培养，提升数字人才有效供给^[2]。由此可见，互联网IT相关岗位需求正在从单一技术开发向数据分析、人工智能、产品运营、平台服务等多方向延伸，对高校学生的专业能力、实践经验和岗位适配能力提出了更高要求。

对于大数据相关专业学生而言，实习岗位不仅是从课堂学习走向企业实践的重要过渡环节，也是了解行业用人需求、判断岗位发展方向和提升就业竞争力的重要信息来源。随着互联网用户规模持续扩大，网络平台已经成为企业发布招聘信息和学生获取实习机会的重要渠道之一。中国互联网络信息中心发布的《中国互联网络发展状况统计报告》表明，我国互联网用户规模和互联网普及水平持续提升，为网络招聘、在线求职和岗位信息传播提供了重要基础^[3]。在这一背景下，招聘平台中的岗位信息具有较高的数据分析价值，其内容通常包含岗位方向、城市、薪资、实习时长、到岗天数、岗位标签和公司信息等字段，能够为分析互联网IT类实习岗位需求提供基础数据。

国内已有研究表明，利用Python网络爬虫采集招聘平台岗位数据，并结合数据清洗和可视化分析方法，可以对薪资水平、热点城市、岗位福利和技能需求等内容进行分析，从而为求职者岗位选择和职业规划提供参考^{[4][5]}。同时，随着大数据在人力资源领域的应用不断深入，招聘系统也逐渐从传统信息发布方式转向多维数据收集、分析和匹配的智能化方向^[6]。因此，围绕招聘平台岗位数据开展采集、清洗、存储和分析，具有一定的现实意义和实践价值。

然而，互联网IT类实习岗位信息主要分散在招聘平台、企业官网和社交媒体等

渠道中，具有更新频率高、岗位数量大、字段格式不统一、信息结构较为分散等特点。若仅依靠人工浏览和手动整理，不仅效率较低，而且难以对大量岗位信息进行持续、系统的统计分析。中国信息通信研究院相关报告也指出，数字经济发展正在持续推动产业数字化和数字产业化进程，对数据资源利用、数字技术应用和数字人才培养提出了更高要求^[7]。因此，有必要借助网络爬虫、数据库存储、数据预处理和可视化分析等技术手段，对互联网 IT 类实习岗位进行自动化采集和结构化处理，将原本分散在网页中的岗位信息转化为可查询、可统计、可分析的数据资源。

本课题以实习僧平台为数据来源，围绕平台“互联网 IT”分类下的实习岗位展开研究，构建一套从菜单任务构建、岗位信息抓取、数据清洗处理、数据库存储到数据提取分析的完整流程。通过该流程，可以将网页中的岗位信息转化为结构化数据，并进一步分析岗位方向分布、城市分布、薪资特征、急招情况和岗位标签等内容，为了解互联网 IT 类实习岗位需求、辅助学生把握实习市场结构和优化岗位选择提供数据支撑。

1.2 课题内容与方法

1.2.1 研究方向

本课题围绕实习僧平台“互联网 IT”类实习岗位的数据采集与分析展开，重点研究岗位信息的自动化获取、结构化存储、数据预处理与后续统计分析。具体研究方向主要包括以下几个方面。

从已有研究来看，国内已有学者以 BOSS 直聘、拉勾网等招聘平台为数据来源，利用 Python 爬虫采集岗位数据，并进一步分析就业数据、岗位分布和技能需求等内容^[8]。也有研究将 Python 爬虫与数据可视化、预测分析相结合，用于构建从网页数据获取到图形化展示的数据分析流程^[9]。因此，本课题选择实习僧平台作为数据来源，围绕“互联网 IT”类实习岗位开展采集、预处理、存储、提取和可视化分析，具有较强的技术可行性和应用关联性。

首先，研究如何利用 Selenium 自动化技术从实习僧平台采集互联网 IT 类实习岗位数据。课题并不是直接对全站岗位进行无差别抓取，而是先从首页“互联网 IT”菜单中提取岗位分类，构造标准化搜索任务，再由主抓取程序进一步访问列表页和详情页，提取岗位标题、城市、薪资、实习要求、岗位标签和公司信息等字段。

其次，研究如何利用 MySQL 对采集任务、岗位详情数据和抓取进度进行结构化管理。系统通过输入任务表保存搜索入口，通过岗位详情表保存原始抓取结果，通过进度表记录任务执行状态和详情抓取状态，从而支持任务去重、结果去重和断点续爬。

再次，研究如何利用 Pandas 和正则表达式对原始岗位数据进行预处理。针对薪资、实习时长、每周到岗天数、岗位标签和急招标识等字段，课题对其进行清洗、转换和结构化构造，生成更适合统计分析的预处理结果表 `processed_job_details`。

最后，研究如何基于预处理结果表进行数据提取和可视化分析。课题使用 SQL 提取岗位方向分布、城市分布、高薪岗位特征和技能关键词等统计结果，并在 Jupyter Notebook 中结合 Pandas、Seaborn、Matplotlib 和 WordCloud 对数据结果进行可视化展示。

同时，针对岗位数据抓取过程中任务量大、分页多、重复抓取风险较高以及程序中断后恢复困难等问题，本课题还研究了在单机环境下通过多进程协同实现伪分布式 Selenium 自动爬虫的方法，并结合 URL 规范化、双标签页抓取、数据库进度记忆和异常处理机制，提高系统运行的稳定性和可恢复性。

1.2.2 要解决的问题及所需信息

本课题主要围绕以下问题展开：

（1）研究互联网 IT 类实习岗位的方向分布情况。该问题需要统计不同岗位方向对应的岗位数量，以分析当前平台中发布数量较高的岗位类别，判断互联网 IT 类实习岗位内部的需求结构。

（2）研究互联网 IT 类实习岗位的城市分布情况。该问题需要统计不同城市对应的岗位数量，以分析岗位资源在不同地区之间的分布特征，判断互联网 IT 实习机会是否集中于北京、上海、深圳、广州、杭州、成都等互联网产业较活跃的城市。

（3）研究高薪岗位通常具备哪些基本特征。该问题需要使用岗位薪资、城市、岗位方向、实习月数和每周到岗天数等信息，对薪资较高的岗位进行筛选和观察，从而分析高薪岗位在岗位属性和实习要求方面的共性。

（4）研究企业对技术能力和岗位标签的关注重点。该问题主要基于结构化后的 `job_tags` 字段，对 Python、SQL、Java、人工智能、数据分析、机器学习、深度学习、AI Agent 等关键词进行统计，同时结合岗位标签词云观察实习福利、工作制度和成长支持等高频标签。

(5) 研究伪分布式爬虫在实际运行中的实现问题。该问题需要解决菜单任务构建、任务去重、详情链接去重、抓取进度记忆、断点续爬、多进程任务协调和异常页面处理等问题，以保证系统能够在单机环境下稳定完成批量岗位数据采集。

为解决上述问题，系统需要采集并维护的主要信息包括：岗位分组名称、岗位方向、搜索链接、详情链接、岗位标题、发布时间、薪资、城市、每周到岗天数、实习时长、岗位标签、公司名称和公司简介等业务字段。同时，系统还需要记录任务执行状态、已完成页码和详情完成记录等进度信息，以支撑任务调度、结果去重和抓取恢复。

1.2.3 解决问题的方法

本课题的数据采集采用“前置任务构建 + 主程序伪分布式抓取”的两阶段方式。前置脚本 `shixiseng_internet_it_mysql_builder.py` 负责从实习僧首页提取“互联网 IT”菜单下的岗位分类，构造标准化搜索任务，并写入输入任务表。主抓取脚本 `shixiseng_optimized_empty_guard.py` 负责从任务表中读取待执行任务，在单机环境下启动多个独立 Selenium 浏览器进程，对搜索结果页和职位详情页进行协同抓取。由于该方案是在同一台主机上通过多进程模拟分布式协作，因此可将其视为伪分布式 Selenium 自动爬虫。

网络爬虫作为一种面向网页资源的信息采集与信息检索技术，能够按照设定规则自动访问网页并提取目标数据，在招聘信息、就业数据和互联网公开数据采集中具有较强适用性^{[10][11]}。与人工逐页浏览相比，基于 Python 的爬虫方法能够提高岗位数据获取效率，也便于后续结合数据库和数据分析工具进行结构化处理。

在抓取过程中，系统通过 URL 规范化减少同义链接带来的重复抓取问题，通过结果表中的联合唯一键控制重复数据写入，通过任务进度表和详情进度表记录抓取状态，从而实现任务去重、详情去重、进度维护和断点续爬。同时，主抓取程序采用“列表页标签 + 详情页标签”的双标签页设计，使列表页翻页和详情页解析相对分离，减少页面频繁跳转导致的状态丢失问题。

数据存储采用 MySQL 关系型数据库完成。系统分别使用输入任务表、原始岗位详情表、预处理结果表和进度表保存任务入口、抓取结果、分析数据和运行状态。其中，原始岗位详情表用于保留网页抓取所得的完整岗位信息，预处理结果表用于保存字段清理和变量构造后的分析型数据，进度表用于支持程序中断后的恢复执行。

数据预处理主要使用 Pandas、正则表达式和数据库连接工具完成。预处理过程

包括删除冗余字段、构造急招标志变量、整理岗位标签字段、提取薪资区间数值、提取实习月数和每周到岗天数等操作。经过处理后，原始文本字段被转换为更适合统计分析的结构化变量。

数据提取与分析主要结合 SQL 和 Jupyter Notebook 完成。第 5 章使用 SQL 从 processed_job_details 表中提取与研究问题相关的统计结果，第 6 章在 Jupyter Notebook 中使用 Pandas、Seaborn、Matplotlib 和 WordCloud 对结果进行可视化展示。通过这一流程，课题实现了从网页数据采集、数据库存储、字段预处理到统计提取和图形展示的完整分析过程。

1.3 课题目标和使用技术

本课题围绕实习僧平台“互联网 IT”类实习岗位，构建一套集数据采集、数据清洗、数据存储、数据提取和可视化分析于一体的处理流程，实现岗位信息的自动化获取与结构化分析。通过该流程，将网页中分散的岗位信息转换为可查询、可统计、可分析的数据资源，并从岗位方向、城市分布、薪资水平、急招状态和岗位标签等角度提取有价值的信息，为了解互联网 IT 类实习岗位需求特征提供参考。

在技术实现方面，本课题以 Python 作为主要开发语言，使用 Selenium 完成网页自动化访问、菜单交互、列表页翻页和岗位详情页抓取，使用 BeautifulSoup 辅助解析页面内容；使用 MySQL 保存菜单任务、岗位详情数据、预处理结果和抓取进度记录；使用 Pandas、正则表达式和 SQL 完成字段清洗、变量构造与统计结果提取；使用 Seaborn、Matplotlib 和 WordCloud 对岗位方向、城市分布、技能关键词、急招情况和岗位标签进行可视化展示。各技术工具在本课题中的作用如表 1.1 所示。

表 1.1 主要技术工具及作用说明表

技术工具	所属环节	主要作用
Python	整体开发	编写数据采集、预处理、数据提取和可视化分析脚本
Selenium	数据采集	实现网页自动访问、菜单交互、列表页翻页和详情页抓取
BeautifulSoup	页面解析	辅助解析网页结构，提取岗位文本和标签信息
MySQL	数据存储	保存菜单任务、岗位详情、预处理结果和抓取进度
Pandas	数据预处理	完成字段清洗、变量构造、分组统计和数据转换
SQL	数据提取	提取岗位方向、城市分布、高薪岗位和关键词统计结果
Seaborn / Matplotlib	可视化分析	绘制岗位数量、城市分布、急招占比等统计图表
WordCloud	文本可视化	生成岗位标签词云，展示高频标签特征

从技术支撑角度看，WebDriver 作为浏览器远程控制接口，能够驱动浏览器执行页面访问、元素交互和行为控制等操作，为 Selenium 自动化采集动态网页数据提供了标准化基础^[12]。在数据处理方面，Pandas 提供了适合表格型数据处理的数据结构，能够支持字段清洗、数据转换和聚合统计等操作^[13]。在数据可视化方面，Matplotlib 可用于生成二维图形和报告型图像^[14]，Seaborn 则进一步提供了面向统计分析的可视化接口^{[15][16]}。因此，本课题所采用的技术工具与岗位数据采集、预处理、统计提取和可视化分析任务具有较高匹配度。

开发环境方面，本课题主要运行于 PyCharm、Anaconda、Jupyter Notebook 和 Navicat Premium 等工具环境中。整体技术路线突出“伪分布式抓取、数据库进度记忆、结构化数据预处理和问题导向数据提取”的思路，使系统不仅能够完成较大规模岗位数据采集，也能够为后续统计分析和报告展示提供稳定、规范的数据基础。

2 数据采集

2.1 数据来源

本课题的数据来源为实习僧招聘平台。与直接采集全站岗位不同，本课题首先聚焦于首页“互联网 IT”菜单中的岗位分类，以此构造搜索任务，再从相应搜索页和详情页中抓取岗位信息。实习僧首页菜单如图 2.1 所示。

前置脚本负责自动打开实习僧首页、关闭首页弹窗、悬停“互联网 IT”一级菜单、提取分组标题及对应岗位名称、构造搜索链接并写入 MySQL。这样做的目的，是先把网页中的分类入口转换为结构化菜单任务，为后续伪分布式 Selenium 爬虫提供统一、可重复调度的输入。



图 2.1 实习僧首页“互联网 IT”菜单网页图

2.2 数据采集方案

本课题采用“菜单任务构建 + 伪分布式详情抓取”的两阶段采集方案。第一阶段侧重把网站菜单转换为可执行任务，第二阶段侧重在单机环境中利用多进程浏览器协同完成批量抓取，两者共同构成完整的数据采集流水线。

在任务构建阶段，程序从实习僧首页菜单中提取岗位分组和岗位名称，为每一个岗位名称生成标准化搜索链接，并将结果写入 `shixiseng_internet_it_links` 表。由于表中设置了 `group_name` 与 `item_text` 的联合唯一约束，即使程序重复运行，也不会重复写入同一岗位分类，从而保证任务源的稳定性。

在主抓取阶段，程序从输入任务表读取菜单任务，并通过 URL 规范化、分页拆分和多进程协作等方式，对岗位详情页进行批量抓取。程序在同一台主机上启动 4 个独立浏览器进程，每个进程内部再固定使用“列表页标签 + 详情页标签”的双标签页模式，因此它在体系结构上属于伪分布式 Selenium 自动爬虫，而不是传统意义上的多机分布式部署。

为了使这一伪分布式抓取方案能够稳定运行，系统在设计时重点加入了抓取进度记忆机制。每个菜单任务执行到哪一页、每条详情链接是否已经成功抓取，都会实时写入数据库；当程序因网络抖动、页面异常或人工中断而停止时，后续重新启动即可从上次完成的位置继续执行，而无需重新遍历全部页面。

除此之外，系统还通过任务 URL 规范化来消除同义链接，通过 (item_text, detail_url) 联合键防止重复抓取，通过空页面识别和失败熔断避免无效任务死循环。正是这些处理逻辑，使得单机多进程协作能够具备接近工程化分布式任务调度的运行效果。

2.3 采集方案实施

2.3.1 菜单任务构建实施过程

菜单任务构建阶段由 shixiseng_internet_it_mysql_builder.py 脚本完成，其核心目标是将实习僧首页“互联网 IT”菜单中的岗位分类信息提取出来，并转换为可供后续主抓取程序直接调用的结构化任务数据。与直接从搜索页开始抓取相比，该阶段的作用在于先建立规范、稳定、可重复调度的任务源，使整个采集流程具备更清晰的入口和更高的可控性。

（1）数据库连接

在数据库连接环节，程序预先配置 MySQL 的主机地址、端口、用户名、密码、数据库名及字符集等参数，并在此基础上建立与 MySQL 服务的连接。若目标数据库尚未创建，程序会自动完成数据库初始化操作。这样处理的意义在于：一方面保证后续任务表能够顺利创建，另一方面也使脚本具备“首次运行即可自动搭建环境”的能力，从而减少人为干预。通过这一部分代码，系统为后续菜单任务的稳定写入奠定了基础。对应内容见图 2.2 与图 2.3。

```

32 # =====
33 # 数据库配置
34 # =====
35 MYSQL_CONFIG = {
36     "host": "127.0.0.1",
37     "port": 3306,
38     "user": "root",
39     "password": "123456",
40     "database": "shixiseng_db",
41     "charset": "utf8mb4"
42 }

```

图 2.2 MySQL 连接参数配置代码图

```

109 def connect(self):
110     """
111     建立 MySQL 数据库连接。如果目标数据库不存在，则自动创建。
112     """
113     try:
114         # 首先连接到 MySQL 服务，不指定数据库，用于检查和创建数据库
115         self.server_conn = mysql.connector.connect(
116             host=self.host,
117             port=self.port,
118             user=self.user,
119             password=self.password,
120             charset=self.charset
121         )
122
123         if self.server_conn.is_connected():
124             server_cursor = self.server_conn.cursor()
125             server_cursor.execute(
126                 f"CREATE DATABASE IF NOT EXISTS `{self.database}` "
127                 f"DEFAULT CHARACTER SET {self.charset}"
128             )
129             server_cursor.close()
130             self.server_conn.close()
131             self.server_conn = None

```

图 2.3 MySQL 数据库自动创建代码图

(2) 数据表创建

在数据库环境准备完成后，程序继续创建输入任务表 `shixiseng_internet_it_links`。该表主要用于保存由首页菜单提取出的岗位分类任务，字段中包含岗位分组名称、岗位文本、搜索链接以及创建时间和更新时间等信息。为了避免程序重复运行时产生重复任务，表结构中设置了 `(group_name, item_text)` 联合唯一约束，使同一分组下的同一岗位分类只会保留一条记录。这一设计使输入任务表不仅具备数据存储功能，也兼具任务源去重与统一管理功能。对应内容见图 2.4 与图 2.5。

```

149     def create_table(self):
150         """
151         创建用于存储实习僧菜单链接的数据表（如果不存在）。
152         保证 group_name 和 item_text 联合唯一。
153         """
154         create_sql = """
155         CREATE TABLE IF NOT EXISTS shixiseng_internet_it_links (
156             id BIGINT PRIMARY KEY AUTO_INCREMENT,
157             group_name VARCHAR(255) NOT NULL,
158             item_text VARCHAR(255) NOT NULL,
159             item_url TEXT NOT NULL,
160             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
161             updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
162                 CURRENT_TIMESTAMP,
163             UNIQUE KEY uniq_group_item (group_name, item_text)
164         ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
165         """
166         try:
167             self.cursor.execute(create_sql)
168             self.conn.commit()
169             print("[日志] 数据表 shixiseng_internet_it_links 检查/创建成功。")
170         except Error as e:
171             raise RuntimeError(f"[错误] 创建数据表失败: {e}")

```

图 2.4 输入任务表建表代码图

```

(base) PS E:\Jupyter_Notebook\课程设计> & E:/anaconda3/envs/reptile/python.exe e:/Jupt
er_Notebook/课程设计/shixiseng_internet_it_mysql_builder.py
[日志] 正在解析 ChromeDriver 路径...
[日志] 成功获取 ChromeDriver 路径: C:\Users\MSIPC\.wdm\drivers\chromedriver\win64\146
.0.7680.153\chromedriver.exe
[日志] MySQL 已成功连接至数据库: shixiseng
[日志] 数据表 shixiseng_internet_it_links 检查/创建成功。

```

图 2.5 数据库连接与建表运行日志图

(3) 浏览器初始化

在浏览器初始化阶段，程序利用 Selenium 启动 Chrome 浏览器，并根据运行环境决定是否启用无头模式。由于招聘网站通常对自动化访问具有一定检测机制，因此程序还通过注入脚本对浏览器中的 WebDriver 特征进行隐藏，以降低被识别为自动化程序的风险。该步骤的意义并不只是“打开网页”，而是为后续首页访问、菜单悬停、元素定位和数据提取提供一个相对稳定的运行环境。对应内容见图 2.6、图 2.7 和图 2.8。

```

227 def __init__(self, driver_path: str, headless: bool = False, wait_seconds: in
228     """初始化 Selenium WebDriver。..."""
236     options = webdriver.ChromeOptions()
237
238     if headless:
239         options.add_argument("--headless=new")
240
241     options.add_argument("--start-maximized")
242     options.add_argument("--disable-blink-features=AutomationControlled")
243     options.add_argument("--lang=zh-CN")
244     options.add_argument("--disable-gpu")
245     options.add_argument("--no-sandbox")
244     options.add_argument("--disable-gpu")
245     options.add_argument("--no-sandbox")
246     options.add_argument("--disable-dev-shm-usage")
247     options.add_argument("--disable-extensions")
248     options.add_argument("--disable-background-networking")
248     options.add_argument("--disable-background-networking")
249     options.add_argument("--disable-sync")
250     options.add_argument("--disable-notifications")
251     options.add_argument("--no-first-run")
252     options.add_argument("--no-default-browser-check")
249     options.add_argument("--disable-sync")
250     options.add_argument("--disable-notifications")
251     options.add_argument("--no-first-run")
252     options.add_argument("--no-default-browser-check")
253     options.add_argument("--log-level=3")
254     options.add_argument("--window-size=1400,900")
255     options.add_argument(
256         "user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
257         "AppleWebKit/537.36 (KHTML, like Gecko) "
258         "Chrome/127.0.0.0 Safari/537.36"
259     )

```

图 2.6 Selenium 浏览器初始化代码图

```

285
286 # 注入脚本，进一步隐藏 WebDriver 特征
287 try:
288     self.driver.execute_cdp_cmd(
289         "Page.addScriptToEvaluateOnNewDocument",
290         {
291             "source": """
292                 Object.defineProperty(navigator, 'webdriver', {
293                     get: () => undefined
294                 });
295             """
296         }
297     )

```

图 2.7 WebDriver 自动化特征隐藏代码图

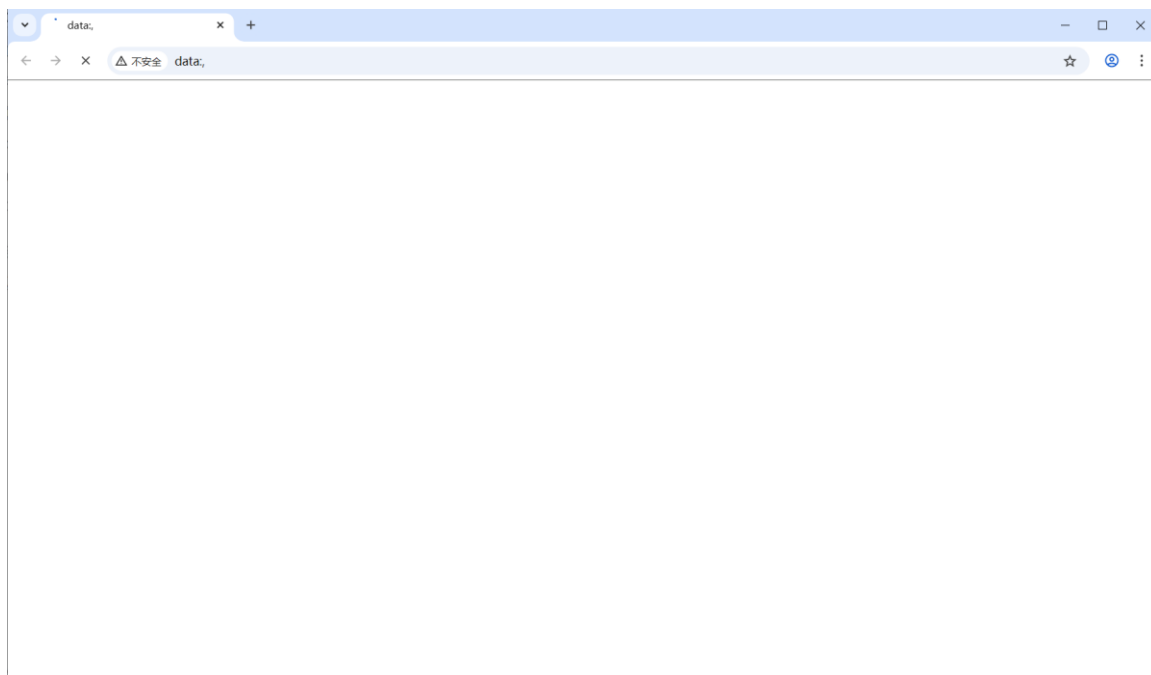


图 2.8 Selenium 浏览器启动结果图

(4) 打开首页与关闭弹窗

在首页访问阶段，程序首先打开实习僧官网首页，并等待页面主体加载完成。由于首页存在弹窗遮挡菜单区域，程序在页面加载后会优先识别并关闭弹窗，以确保“互联网 IT”菜单可以正常展开和交互。该环节虽然在实现上较为基础，但在实际运行中十分关键；若不先处理弹窗，后续菜单悬停和岗位项提取都可能失败。因此，该步骤本质上属于采集流程中的前置清障过程。对应内容见图 2.9、图 2.10 和图 2.11。

```
300 def open_homepage(self):  
301     """  
302     打开实习僧官方首页。  
303     """  
304     self.driver.get(self.base_url)  
305     print(f"[日志] 已成功打开实习僧首页: {self.base_url}")
```

图 2.9 实习僧首页访问代码图

```

307 def close_home_popup(self):
308     """
309     尝试关闭首页可能出现的广告或活动弹窗。
310     """
311     try:
312         time.sleep(2)
313         close_btn = self.wait.until(
314             EC.element_to_be_clickable((By.CSS_SELECTOR, "div.openmask img.closeOpen"))
315         )
316
317         self.driver.execute_script(
318             "arguments[0].scrollIntoView({block:'center', inline:'center'});",
319             close_btn
320         )
321         time.sleep(0.3)
322
323         try:
324             close_btn.click()
325         except Exception:
326             self.driver.execute_script("arguments[0].click();", close_btn)
327
328         self.wait.until(
329             EC.invisibility_of_element_located((By.CSS_SELECTOR, "div.openmask"))
330         )
331         print("[日志] 首页弹窗已成功关闭。")
332
333     except Exception:
334         print("[日志] 未检测到首页弹窗或无需关闭。")

```

图 2.10 首页弹窗关闭处理代码图



图 2.11 首页弹窗关闭结果图

(5) 菜单悬停与岗位链接提取

完成首页准备后，程序通过 Selenium 的鼠标悬停操作展开“互联网 IT”一级菜单，并进一步获取其下属的岗位分组和岗位项信息。在解析过程中，程序按照“分组标题—岗位名称”的层次结构进行遍历，将网页中的非结构化菜单内容转换为程序可以识别的任务对象。与此同时，为避免同一岗位入口因页面结构重复或重复遍历而被多次记录，程序在提取阶段引入集合去重机制，对岗位项进行唯一性控制。该环节标志着系统已经将网页展示层中的菜单入口成功转化为可用于后续抓取的任务列表。对应内容见图 2.12、图 2.13、图 2.14 和图 2.15。

```

344 internet_it = self.wait.until(
345     EC.presence_of_element_located((
346         By.XPATH,
347         "//*[@div[contains(@class,'type-item')]]//a[contains(@class,'type-item-first') and normalize-space()='互联网IT']"
348     ))
349 )
350
351 self.driver.execute_script(
352     "arguments[0].scrollIntoView({block:'center', inline:'center'});",
353     internet_it
354 )
355 time.sleep(0.5)
356
357 self.actions.move_to_element(internet_it).pause(1).perform()
358 print("[日志] 鼠标已悬停至“互联网IT”菜单。")
359
360 parent = self.wait.until(
361     EC.presence_of_element_located((
362         By.XPATH,
363         "//*[@div[contains(@class,'type-item')]]//a[contains(@class,'type-item-first') and normalize-space()='互联网IT']"
364     ))
365 )
366
367 type_list = parent.find_element(By.CSS_SELECTOR, "div.type-list")

```

图 2.12 “互联网 IT”菜单悬停展开代码图



图 2.13 “互联网 IT”菜单展开页面图

```

395 def collect_grouped_links(self, parent_elem) -> list:
396     """ """
407     results = []
408     raw_candidate_count = 0
409     duplicate_url_count = 0
410     # 基于 item_url 在采集阶段完成首次出现保留的去重，避免重复链接继续进入结果列表或数据库。
411     # 这样可防止菜单结构重复、别名岗位或页面结构异常导致的重复链接入库问题。
412     seen_item_urls = set()
413
414     try:
415         type_list = parent_elem.find_element(By.CSS_SELECTOR, "div.type-list")
416         groups = type_list.find_elements(By.XPATH, "./div")
417
418         for group_idx, group in enumerate(groups, start=1):
419             try:
420                 group_title_elem = group.find_element(By.CSS_SELECTOR, "a.first-text.text")
421                 group_title = group_title_elem.text.strip()
422             except Exception:
423                 group_title = f"未命名分组_{group_idx}"
424
425             all_links = group.find_elements(By.CSS_SELECTOR, "a.text")
426
427             for link in all_links:
428                 classes = (link.get_attribute("class") or "").strip()
429                 text = (link.text or "").strip()
430
431                 # 排除空文本和一级标题本身
432                 if not text or "first-text" in classes:
433                     continue
434
435                 # 构造标准化搜索 URL，并在采集阶段按 item_url 执行去重。
436                 item_url = self.build_search_url(text)
437                 raw_candidate_count += 1
438
439                 if item_url not in seen_item_urls:
440                     seen_item_urls.add(item_url)
441                     results.append({
442                         "group_name": group_title,
443                         "item_text": text,
444                         "item_url": item_url
445                     })
446                 else:
447                     duplicate_url_count += 1

```

图 2.14 岗位分组链接提取与去重代码图

```

• (base) PS E:\Jupyter_Notebook\课程设计> & E:/anaconda3/envs/reptile/python.exe e:/J
upter_Notebook/课程设计/shixiseng_internet_it_mysql_builder.py
[日志] 正在解析 ChromeDriver 路径...
[日志] 成功获取 ChromeDriver 路径: C:\Users\MSIPC\wdm\drivers\chromedriver\win64\
146.0.7680.153\chromedriver.exe
[日志] MySQL 已成功连接至数据库: shixiseng
[日志] 数据表 shixiseng_internet_it_links 检查/创建成功。
[日志] 已成功打开实习僧首页: https://www.shixiseng.com/
[日志] 首页弹窗已成功关闭。
[日志] 鼠标已悬停至“互联网IT”菜单。
[日志] “互联网IT”子菜单已完全展开
[日志] 菜单链接采集完成: 分组数=12, 去重前数量=90, 去重后数量=89, 去掉掉数量=1。

```

图 2.15 岗位分组链接去重运行日志图

(6) 构造搜索 URL 并写入数据库

在提取到岗位文本之后，程序根据岗位名称构造标准化搜索 URL，并将分组名称、岗位文本和对应链接统一写入数据库。这样，每一条菜单记录都从网页中的一个静态入口转变为数据库中的一个标准化搜索任务。经过这一处理，输入任务表中的数据已经不再是简单的菜单展示结果，而是后续主抓取程序可以直接读取、分页访问和重复调度的任务源。至此，菜单任务构建阶段完成。对应内容见图 2.16、图 2.17 和图 2.18。

```

379 def build_search_url(self, keyword_text: str) -> str:
380     """根据岗位关键词构造标准化的实习搜索页 URL。..."""
381     encoded_keyword = quote(keyword_text)
382     return (
383         "https://www.shixiseng.com/interns"
384         f"?keyword={encoded_keyword}&city=%E5%85%A8%E5%98%BD&type=intern&from=menu"
385     )
386
387
388
389
390
391
392
393

```

图 2.16 标准化搜索 URL 构造代码图

```

522 # 3. 提取与处理数据
523 rows = spider.collect_grouped_links(parent_menu)
524
525 # 4. 打印并入库
526 spider.print_results(rows)
527 db.upsert_rows(rows)

```

图 2.17 菜单任务写入数据库代码图

对象: shixiseng_internet_it_links @shixisen...

开始任务 文本 筛选 排序 导入 导出数据生成 创建图表

id	group_name	item_text	item_url	created_at	updated_at
1	产品	产品	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
2	产品	产品经理	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
3	产品	产品助理	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
4	产品	产品设计	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
5	产品	产品项目管理	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
6	产品	用户研究	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
7	运营	运营	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
8	运营	运营助理	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
9	运营	运营专员	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
10	运营	产品运营	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
11	运营	内容运营	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
12	运营	SEO/SEM运营	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
13	运营	电商运营	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
14	运营	活动运营	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
15	运营	数据运营	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
16	运营	社群运营	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
17	运营	新媒体运营	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
18	运营	游戏运营	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
19	运营	海外运营	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
20	运营	品类运营	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
21	运营	用户运营	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
22	运营	网站运营	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
23	运营	文案编辑	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
24	技术管理	技术管理	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
25	技术管理	架构师	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
26	后端开发	后端开发	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
27	后端开发	C/C++	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
28	后端开发	Java	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
29	后端开发	ERP技术开发	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56
30	后端开发	PHP	https://www.shixiseng.com	2026-03-21 03:03:52	2026-04-08 01:09:56

图 2.18 输入任务表示例数据图

2.3.2 主抓取程序实施过程

主抓取阶段由 `shixiseng_optimized_empty_guard.py` 脚本完成，其主要任务是从输入任务表中读取菜单任务，分页访问岗位列表页，并进一步打开职位详情页提取所需字段，最终将结果写入数据库。与前置任务构建阶段相比，这一阶段不仅承担岗位数据的主体采集工作，还承担了任务调度、去重控制、异常恢复、进度维护和断点续爬等工程化功能，因此是整个采集系统的核心部分。根据程序结构，其实现过程可分为数据库初始化、URL 规范化、浏览器与双标签页初始化、数据库管理封装、列表页处理、详情页字段提取以及多进程协同抓取等几个环节。

(1) 数据库配置与初始化

在数据库配置与初始化环节，程序统一定义主抓取阶段所需的数据库连接参数，并通过 `MySQLManager` 类集中管理数据库连接与表结构维护。与前置脚本仅创建任

务表不同，主抓取程序需要同时维护输入任务表、结果表、任务进度表和详情进度表。其中，输入任务表提供待抓取的菜单任务，结果表用于保存解析后的岗位详情数据，任务进度表记录每个搜索任务已完成到哪一页，详情进度表则记录哪些详情页已经成功抓取。特别是结果表中采用 (item_text, detail_url) 联合唯一键进行约束，使同一岗位分类下的同一详情链接不会被重复写入，从数据库层面保证了结果唯一性。对应内容见图 2.19。

```
44 # =====
45 # 全局数据库配置
46 # =====
47 MYSQL_CONFIG = {
48     "host": "127.0.0.1",
49     "port": 3306,
50     "user": "root",
51     "password": "123456",
52     "database": "shixiseng_db",
53     "charset": "utf8mb4",
54 }
55
56 # 输入任务表，由前置菜单构造脚本生成。
57 INPUT_TASK_TABLE = "shixiseng_internet_it_links"
58
59 # 最终详情结果表。
60 OUTPUT_RESULT_TABLE = "shixiseng_job_details"
61
62 # 任务进度表，记录每个菜单任务已完成到第几页。
63 TASK_PROGRESS_TABLE = "shixiseng_task_progress"
64
65 # 详情进度表，记录哪些 (item_text, detail_url) 已成功抓取。
66 DETAIL_PROGRESS_TABLE = "shixiseng_detail_progress"
```

图 2.19 主抓取程序数据库配置与表结构定义代码图

(2) URL 规范化与去重

在 URL 规范化与去重环节，程序分别对任务搜索链接和职位详情链接进行标准化处理。对于搜索 URL，程序会统一协议、域名和路径形式，并对关键查询参数进行清洗与保留，使语义相同但书写形式不同的链接能够被识别为同一任务；对于详情 URL，则进一步去除无关的 query 参数和 fragment 片段，以确保同一岗位详情页在系统内部对应唯一的标准链接。通过这一处理，程序有效避免了由于链接格式差异带来的重复抓取问题，也为数据库联合键去重和历史抓取记录判断提供了统一依据。对应内容见图 2.20、图 2.21 和图 2.22。

```

97 def canonicalize_search_url(item_url: str, page_no: int = None, keep_page: bool = False) -> str:
98     """..."""
106
107     if not item_url:
108         return SEARCH_BASE_URL
109
110     parsed = urlparse(item_url.strip())
111
112     scheme = parsed.scheme or "https"
113     netloc = parsed.netloc or "www.shixiseng.com"
114     path = parsed.path or "/interns"
115
116     if "shixiseng.com" not in netloc:
117         netloc = "www.shixiseng.com"
118
119     if path != "/interns":
120         path = "/interns"
121
122     raw_pairs = parse_qs(parsed.query, keep_blank_values=True)
123     query_map = {}
124     for k, v in raw_pairs:
125         k = (k or "").strip()
126         if k not in ALLOWED_SEARCH_PARAMS:
127             continue
128         v = normalize_text_param(v) # 规范化文本参数，避免编码差异导致同义 URL 不一致。
129         if not v:
130             continue
131         query_map[k] = v
132
133     if page_no is not None:
134         page_no = max(1, int(page_no))
135         if page_no > 1:
136             query_map["page"] = str(page_no)
137         else:
138             query_map.pop("page", None)
139     else:
140         if not keep_page:
141             query_map.pop("page", None)
142         else:
143             page_val = (query_map.get("page") or "").strip()
144             if not page_val.isdigit() or int(page_val) <= 1:
145                 query_map.pop("page", None)
146             else:
147                 query_map["page"] = str(int(page_val))
148
149     ordered_pairs = []
150     for key in ALLOWED_SEARCH_PARAMS:
151         if key in query_map:
152             ordered_pairs.append((key, query_map[key]))
153
154     canonical_query = urlencode(ordered_pairs, doseq=False)
155     return urlunparse((scheme, netloc, path, "", canonical_query, ""))

```

图 2.20 任务 URL 标准化处理代码图

```

163 def normalize_detail_url(detail_url: str) -> str:
164     """规范化详情页 URL，去掉 query 和 fragment，统一绝对路径。"""
165     if not detail_url:
166         return ""
167     detail_url = detail_url.strip()
168
169     if detail_url.startswith("//"):
170         detail_url = "https:" + detail_url
171     elif detail_url.startswith("/"):
172         detail_url = "https://www.shixiseng.com" + detail_url

```

图 2.21 详情页 URL 规范化处理代码图

```

[2026-04-22 23:36:59] [Worker-1] 初始化初始任务 23/23 的页面共享队列
[2026-04-22 23:36:59] [Worker-1] 初始化页面任务，先探测真实总页数：https://www.shixiseng.com/interns?keyword=%E8%BF%90%E8%90%A5%E5%8A%A9%E7%90%86&city=%E5%85%A8%E5%9B%BD&type=intern&from=menu
[2026-04-22 23:37:11] [Worker-2] 已将页面任务加入共享池：运营 | 网站运营 | 从第 1 页到第 294 页，共 294 页
[2026-04-22 23:37:11] [Worker-2] 页面协作任务：产品 | 产品 | 第 2/202 页
[2026-04-22 23:37:12] [Worker-4] 当前页解析到详情数：20 | 产品 | 产品 | 第 1 页
[2026-04-22 23:37:12] [Worker-4] 已存在，跳过详情：1/20 产品 | https://www.shixiseng.com/intern/inn_srfaxrabvzb3
[2026-04-22 23:37:12] [Worker-4] 已存在，跳过详情：2/20 产品 | https://www.shixiseng.com/intern/inn_jem097e8yksq
[2026-04-22 23:37:12] [Worker-4] 已存在，跳过详情：3/20 产品 | https://www.shixiseng.com/intern/inn_ibapshzpn1vp
[2026-04-22 23:37:12] [Worker-4] 已存在，跳过详情：4/20 产品 | https://www.shixiseng.com/intern/inn_s77Lyuyx7miy
[2026-04-22 23:37:12] [Worker-4] 抓详情：5/20 产品 | https://www.shixiseng.com/intern/inn_0lrexz0o99ki
[2026-04-22 23:37:13] [Worker-1] 已将页面任务加入共享池：运营 | 运营助理 | 从第 1 页到第 344 页，共 344 页
[2026-04-22 23:37:13] [Worker-1] 页面协作任务：产品 | 产品 | 第 3/202 页
[2026-04-22 23:37:14] [Worker-3] 已将页面任务加入共享池：运营 | 新媒体运营 | 从第 1 页到第 294 页，共 294 页
[2026-04-22 23:37:14] [Worker-3] 初始化初始任务 21/22 的页面共享队列
[2026-04-22 23:37:14] [Worker-3] 初始化页面任务，先探测真实总页数：https://www.shixiseng.com/interns?keyword=%E8%BF%90%E8%90%A5%E5%8A%A9%E7%90%86&city=%E5%85%A8%E5%9B%BD&type=intern&from=menu

```

图 2.22 URL 历史抓取记录校验图

(3) 浏览器初始化与双标签页设计

在浏览器初始化与双标签页设计环节，程序通过 Selenium 启动浏览器实例，并同样采用隐藏自动化特征的方式提高访问稳定性。与一般的单标签页爬取不同，该程序为每个进程固定分配两个标签页：一个用于列表页访问与翻页，另一个用于职位详情页打开与字段提取。这样设计的优势在于，列表页始终保持在当前分页上下文中，不需要因为进入详情页而频繁回退；详情页标签则可以独立加载岗位信息，实现列表与详情两个操作链路的相对解耦。该机制显著减少了页面跳转导致的状态丢失、回退失败和元素失效问题，是本程序能够稳定运行的重要原因之一。对应内容见图 2.23、图 2.24 和图 2.25。

```

723 class ShiXiSengSeleniumSpider:
724     """封装实习僧列表页和详情页的浏览器操作。"""
725
726     * 墨染止 *
727     def __init__(self, driver_path, headless=False, wait_seconds=15, window_rect=None):
728         startup_begin = monotonic_now()
729         options = webdriver.ChromeOptions()
730         options.page_load_strategy = "eager"

```

图 2.23 主抓取程序浏览器初始化代码图

```

804 self.list_tab = self.driver.current_window_handle
805 self.detail_tab = None # 初始化详情标签页。
806 self.switch_to_list_tab() # 切换到列表标签页。

```

图 2.24 双标签页初始化代码图



图 2.25 双标签页运行界面图

(4) 数据库管理类

在数据库管理类设计方面，MySQLManager 不仅负责数据库连接和建表，还封装了任务读取、结果写入、进度保存和已完成详情判断等一系列关键操作。例如，程序可先从输入任务表中读取待执行的菜单任务，再结合历史进度表判断该任务应从哪一页继续；在详情抓取完成后，又会将解析结果写入结果表，并同步更新详情进度表，从而形成“抓取结果入库—抓取状态落库”的闭环。通过将这些数据库操作集中封装，程序在多进程环境下能够以较统一的方式维护任务状态和结果数据，降低逻辑分散带来的维护难度。对应内容见图 2.26、图 2.27、图 2.28、图 2.29、图 2.30 和图 2.31。

```

330     def connect(self):
331         try:
332             self.conn = mysql.connector.connect(**self.config)
333             self.cursor = self.conn.cursor(dictionary=True)
334             log(f"MySQL 已连接: {self.config.get('database')}")
335         except Error as e:
336             raise RuntimeError(f"MySQL 连接失败: {e}")

```

图 2.26 结果表与进度表建表代码图

```

359     def create_tables(self):
360         """创建程序运行所需的数据表，并统一修正唯一索引。"""
361
362         create_job_details_sql = f"""
363         CREATE TABLE IF NOT EXISTS '{OUTPUT_RESULT_TABLE}' ( ... ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
364         """
365
366         create_task_progress_sql = f"""
367         CREATE TABLE IF NOT EXISTS '{TASK_PROGRESS_TABLE}' ( ... ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
368         """
369
370         create_detail_progress_sql = f"""
371         CREATE TABLE IF NOT EXISTS '{DETAIL_PROGRESS_TABLE}' ( ... ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
372         """
373
374         self.cursor.execute(create_job_details_sql)
375         self.cursor.execute(create_task_progress_sql)
376         self.cursor.execute(create_detail_progress_sql)
377         self.conn.commit()
378
379         self._ensure_result_table_unique_index()
380         self._ensure_detail_progress_unique_index()
381
382         log("MySQL 数据表已就绪。")

```

图 2.27 输入任务读取与去重处理代码图

```

501 def load_input_tasks(self):
502     """ """
505     sql = f"""
506     SELECT DISTINCT ... FROM missing_value
507     """
508     self.cursor.execute(sql)
509     rows = self.cursor.fetchall()
510
511     normalized = []
512     seen = set()
513
514     for row in rows:
515         group_name = (row.get("group_name") or "").strip()
516         item_text = (row.get("item_text") or "").strip()
517         item_url = canonical_task_url(row.get("item_url") or "")
518
519         key = (group_name, item_text, item_url)
520         if key in seen:
521             continue
522         seen.add(key)
523
524         normalized.append(...)
525
526     return normalized

```

图 2.28 岗位详情结果写入数据库代码图

```

600 def upsert_result_row(self, row):
601     """ """
605     sql = f""" ... """
606
607     values = (
608         ...
609     )
610
611     self.cursor.execute(sql, values)
612     self.conn.commit()

```

图 2.29 详情抓取进度写入代码图

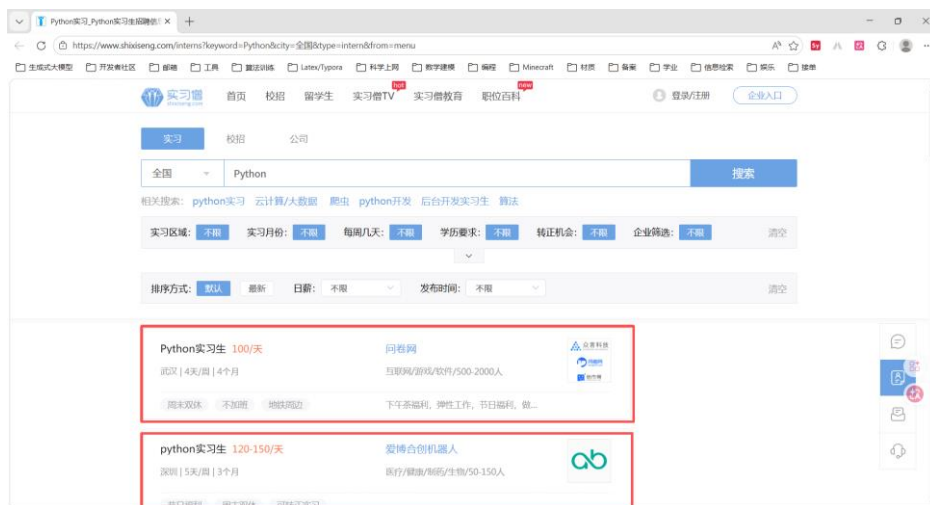


图 2.30 职位详情页字段提取代码图

```

698 def upsert_detail_progress(self, detail_url, group_name, item_text, item_url):
699     """
700     将单条 (item_text, detail_url) 的完成状态写入详情进度表。
701     """
702     detail_url = normalize_detail_url(detail_url)
703     item_url = canonical_task_url(item_url)
704     sql = f"""
705     INSERT INTO `missing_value` ...
706     """
707
708     self.cursor.execute(
709         sql,
710         (detail_url, group_name, item_text, item_url, now_str())
711     )
712
713     self.conn.commit()

```

图 2.31 数据库任务管理与结果维护代码图

(5) 列表页任务处理

在列表页任务处理环节，程序按照分页结果逐页解析岗位列表，并从每条岗位卡片中提取详情链接等基础信息。对每一页而言，程序首先判断页面是否有效、是否存在岗位卡片；若页面为空、结构异常或解析失败，则会结合预设策略进行异常处理，例如跳过空页、记录失败信息或将任务重新加入等待队列。对于正常页面，程序则会把解析出的 (item_text, detail_url) 等信息加入共享字典或共享状态结构，用于多进程之间的去重协调，防止多个进程对同一详情页重复抓取。由此可见，列表页处理不仅承担“提取链接”的作用，更承担着“分页调度入口”和“详情任务分发前筛选”的作用。对应内容见图 2.32、图 2.33、图 2.34、图 2.35 和图 2.36。

```
1621 def process_single_page_task(  
1622     worker_id: int,  
1623     page_task: dict,  
1624     spider,  
1625     db,  
1626     shared_done_detail_dict: dict,  
1627     shared_detail_lock,  
1628     shared_task_runtime: dict,  
1629     shared_task_runtime_lock  
1630 ) -> bool:
```

图 2.32 单页任务处理函数代码图

```
1676 page_items = spider.parse_list_page_links(  
1677     group_name=group_name,  
1678     item_text=item_text,  
1679     item_url=item_url,  
1680     current_page=current_page  
1681 )
```

图 2.33 列表页岗位链接解析代码图

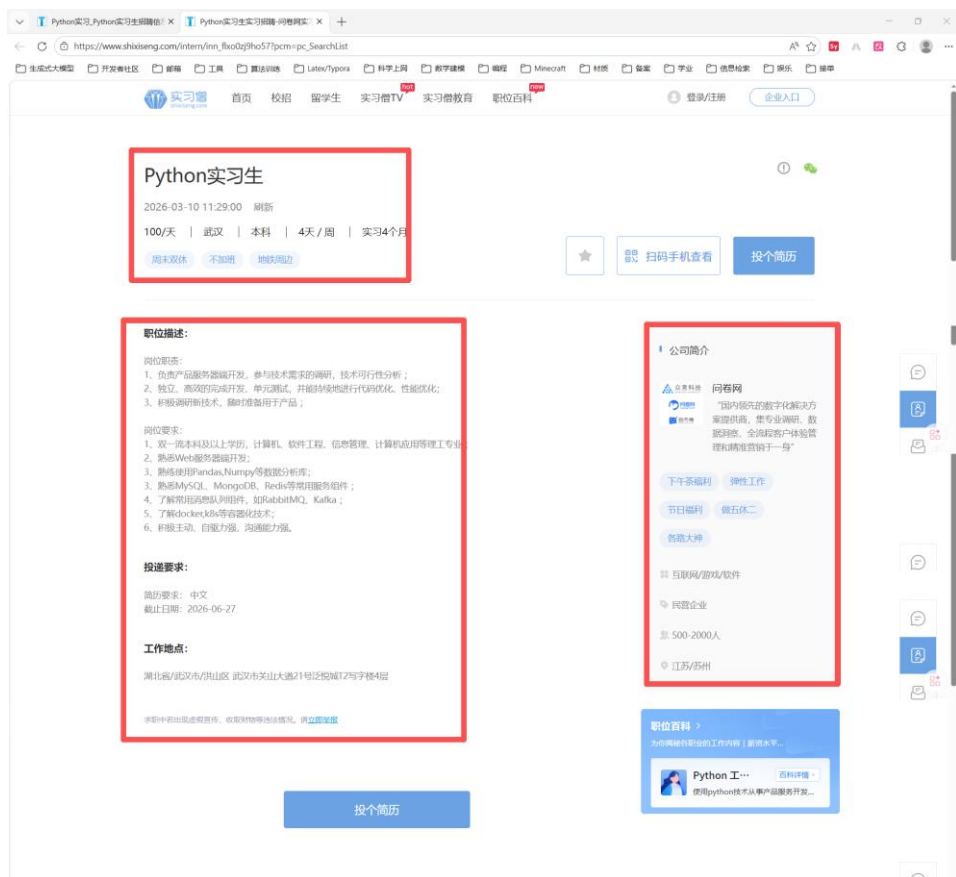


图 2.34 职位详情页页面图

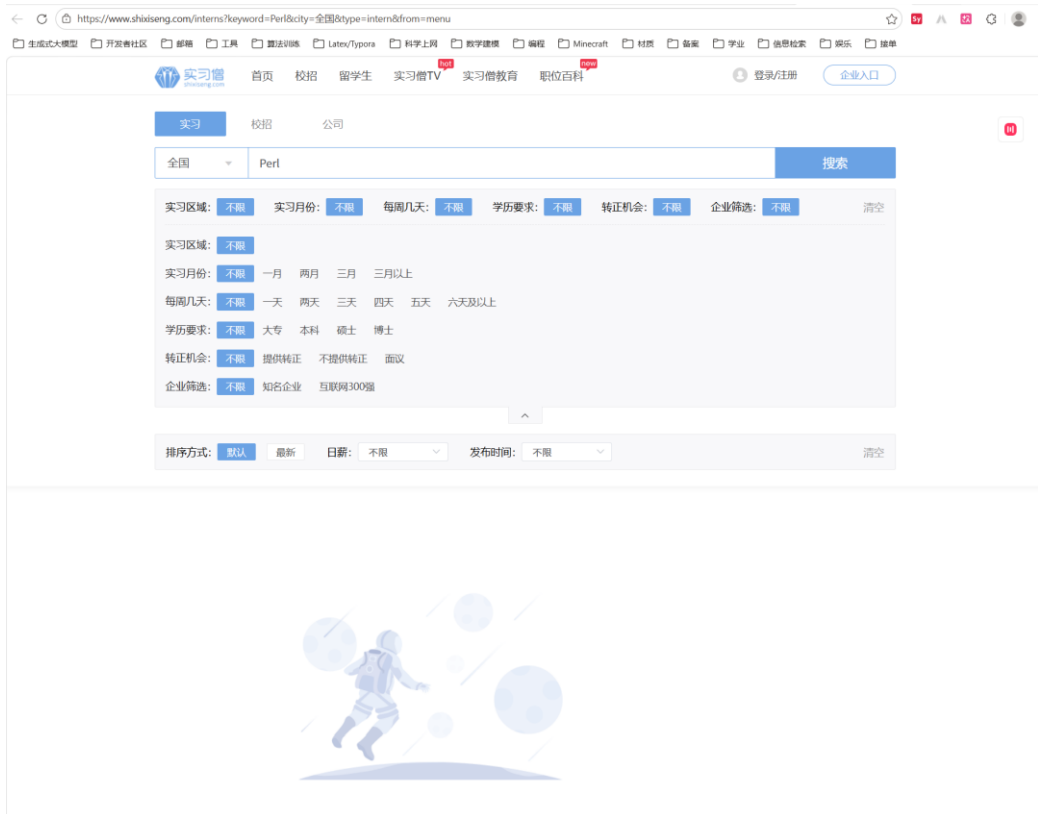


图 2.35 异常空白详情页示意图

```
问题  输出  调试控制台  终端
[2026-04-22 19:58:04] [Worker-1] 初始化页面任务，先探测真实总页数: https://www.shixiseng.com/interns?keyword=%E5%AE%A2%E6%9C%8D&city=%E5%85%A8%E5%9B%BD&type=intern&from=menu
[2026-04-22 19:58:08] [Worker-4] 已将页面任务加入共享池: 客服 | 售前客服 | 从第 1 页到第 26 页, 共 26 页
[2026-04-22 19:58:08] [Worker-4] 初始化初始任务 11/22 的页面共享队列
[2026-04-22 19:58:08] [Worker-4] 初始化页面任务，先探测真实总页数: https://www.shixiseng.com/interns?keyword=%E7%94%B5%E8%AF%9D%E5%AE%A2%E6%9C%8D&city=%E5%85%A8%E5%9B%BD&type=intern&from=menu
[2026-04-22 19:58:09] [Worker-3] 初始化页面任务失败 (状态=timeout): 后端开发 | Perl | 累计失败 1/3
[2026-04-22 19:58:09] [Worker-3] 任务重新放回共享池: 后端开发 | Perl | 原因: 初始任务初始化失败, 等待其他窗口后续接力
[2026-04-22 19:58:09] [Worker-3] 初始化初始任务 10/22 的页面共享队列
[2026-04-22 19:58:09] [Worker-3] 初始化页面任务，先探测真实总页数: https://www.shixiseng.com/interns?keyword=%E5%90%8E%E7%AB%AF%E5%BC%80%E5%8F%91&city=%E5%85%A8%E5%9B%BD&type=intern&from=menu
[2026-04-22 19:58:17] [Worker-1] 已将页面任务加入共享池: 客服 | 客服 | 从第 1 页到第 25 页, 共 25 页
[2026-04-22 19:58:17] [Worker-1] 初始化初始任务 12/23 的页面共享队列
[2026-04-22 19:58:17] [Worker-1] 初始化页面任务，先探测真实总页数: https://www.shixiseng.com/interns?keyword=%E7%BD%91%E7%BB%9C%E5%AE%A2%E6%9C%8D&city=%E5%85%A8%E5%9B%BD&type=intern&from=menu
[2026-04-22 19:58:20] [Worker-2] 初始化页面任务失败 (状态=timeout): 后端开发 | Ruby | 累计失败 1/3
[2026-04-22 19:58:20] [Worker-2] 任务重新放回共享池: 后端开发 | Ruby | 原因: 初始任务初始化失败, 等待其他窗口后续接力
[2026-04-22 19:58:20] [Worker-2] 初始化初始任务 11/22 的页面共享队列
[2026-04-22 19:58:20] [Worker-2] 初始化页面任务，先探测真实总页数: https://www.shixiseng.com/interns?keyword=%E5%AE%A2%E6%9C%8D%E4%B8%93%E5%91%98&city=%E5%85%A8%E5%9B%BD&type=intern&from=menu
[2026-04-22 19:58:21] [Worker-4] 已将页面任务加入共享池: 客服 | 电话客服 | 从第 1 页到第 25 页, 共 25 页
[2026-04-22 19:58:21] [Worker-4] 初始化初始任务 12/22 的页面共享队列
[2026-04-22 19:58:21] [Worker-4] 初始化页面任务，先探测真实总页数: https://www.shixiseng.com/interns?keyword=%E5%A4%A7%E6%95%B0%E6%8D%AE%E5%BC%80%E5%8F%91%E5%B7%A5%E7%A8%8B%E5%B8%88&city=%E5%85%A8%E5%9B%BD&type=intern&from=menu
```

图 2.36 解析失败终端日志图

(6) 详情页字段解析

在详情页字段解析环节，程序进入职位详情页后，会进一步提取岗位标题、发布时间、薪资、工作城市、学历要求、每周到岗天数、实习时长、职位描述、公司名称、公司简介等关键业务字段，并对部分结构化标签或图片链接进行清洗和整理。经过这一阶段处理，原本存在于网页中的半结构化或非结构化招聘信息被转换为能够直接写入数据库字段的结构化结果，为后续的数据清洗、SQL 提取和统计分析提供了数据基础。该步骤决定了采集数据的完整性与可分析性，因此是主抓取程序中最直接面向业务目标的实现环节。对应内容见图 2.37 和图 2.38

```

1047 def _extract_detail_from_html(self, soup):
1048     result = {
1049         "job_title_detail": "",
1050         "publish_time_detail": "",
1051         "salary_detail": "",
1052         "city_detail": "",
1053         "degree_detail": "",
1054         "days_per_week_detail": "",
1055         "intern_months_detail": "",
1056         "job_tags_detail": "",
1057         "job_header_img_links": "",
1058         "job_description": "",
1059         "resume_language": "",
1060         "deadline_detail": "",
1061         "job_address": "",
1062         "company_name_detail": "",
1063         "company_desc_detail": "",
1064         "company_tags_detail": "",
1065         "industry_detail": "",
1066         "company_nature_detail": "",
1067         "company_size_detail": "",
1068         "company_location_detail": "",
1069         "company_intro_img_links": ""
1070     }

```

图 2.37 详情页字段解析代码图

publish_time_detail	salary_detail	city_detail	degree_detail	days_per_week_detail	intern_months_detail	job_tags_detail	job_header_img_links	job_description
2026-01-06 22:29:06	150-300/天	南京	本科	4天 / 周	实习3个月	可转正实习、接受大一大二、		岗位职责：1、负责产品线
2026-03-18 17:30:13	薪资面议	上海	本科	5天 / 周	实习3个月	周末双休	https://sxsimg.xiaoyuanzf	招聘对象：海内外2027f
2026-03-18 17:30:13	薪资面议	上海	本科	5天 / 周	实习3个月	周末双休	https://sxsimg.xiaoyuanzf	招聘对象：海内外2027f
2026-03-20 14:14:11	120-150/天	北京	本科	5天 / 周	实习3个月	学习机会多、成长空间大、		岗位职责：1、协助设计
2026-03-09 11:20:00	120-140/天	广州	本科	4天 / 周	实习3个月	地铁周边、周末双休、一对-		岗位职责：协助产品负
2026-03-18 17:30:10	薪资面议	上海	本科	5天 / 周	实习3个月	周末双休	https://sxsimg.xiaoyuanzf	招聘对象：海内外2027f
2026-03-18 17:30:10	薪资面议	上海	本科	5天 / 周	实习3个月	周末双休	https://sxsimg.xiaoyuanzf	招聘对象：海内外2027f
2026-03-14 00:00:36	150-180/天	上海	本科	5天 / 周	实习6个月	地铁周边、零基础实习、一		a、辅助设计师完成产
2024-06-19 13:53:52	100/天	济南	大专	5天 / 周	实习3个月	暑期实习、零基础实习		岗位职责：1、产品运营
2026-03-12 14:53:46	100-200/天	上海	本科	5天 / 周	实习3个月	不加班、远程实习、节日福		1、根据公司发展需要，
2025-11-06 13:58:07	150/天	成都	本科	5天 / 周	实习6个月	互联网公司、福利好		岗位职责：1、常规定
2026-03-17 14:43:04	200-250/天	北京	本科	5天 / 周	实习6个月	实习证明、晚餐补贴、免费		岗位职责：(具体工作内
2025-09-30 16:09:14	150-200/天	北京	本科	5天 / 周	实习6个月	AI产品、1v1带教、跨团队		(我们期待与你一起探
2023-07-10 16:08:44	100-200/天	杭州	本科	4天 / 周	实习3个月	产品好、领导nice		工作职责：1.调研、分
2026-03-17 15:14:26	350/天	上海	本科	4天 / 周	实习4个月	团队氛围好、晋升空间大、		职位描述：日常实习：面
2024-06-19 13:53:17	150-200/天	济南	本科	5天 / 周	实习3个月	可转正实习		岗位职责：1.竞品分析；
2023-10-16 10:46:36	100-150/天	武汉	本科	5天 / 周	实习3个月	定期团建		工作职责：1.负责海外Sa
2026-03-12 17:33:54	200/天	上海	本科	5天 / 周	实习3个月	餐补、周末双休		职位描述：负责得物交易
2026-03-17 15:13:58	350/天	上海	本科	4天 / 周	实习4个月	团队氛围好、弹性工作、租		职位描述：日常实习：面
2024-03-07 16:13:56	150-250/天	上海	本科	4天 / 周	实习6个月	接受大一大二、留学生实习、		【工作内容】：1、Zer
2026-03-17 16:22:06	250-300/天	北京	本科	5天 / 周	实习3个月	一对一带教		工作职责：参与百度文
2026-03-08 20:00:26	260-310/天	上海	本科	5天 / 周	实习3个月	弹性工作、免费三餐、扁平		工作职责：1.深度调研
2026-03-17 15:13:32	350/天	上海	本科	4天 / 周	实习4个月	团队氛围好、晋升空间大、		职位描述：日常实习：面
2023-05-02 15:24:21	100-200/天	北京	硕士	5天 / 周	实习3个月	暑期实习		一、工作职责：1.主要
2026-01-22 10:22:23	200-250/天	上海	本科	5天 / 周	实习3个月	餐补、地铁周边、周末双休、		岗位职责：1.参与AI辅助
2026-01-22 10:22:23	200-250/天	上海	本科	5天 / 周	实习3个月	餐补、地铁周边、周末双休、		岗位职责：1.参与AI辅助
2026-02-28 14:14:46	150-250/天	深圳	本科	5天 / 周	实习6个月	地铁周边、周末双休		岗位职责：1、对友商C

图 2.38 岗位详情抓取结果图

(7) 多进程 Worker

在多线程 Worker 协同抓取环节，程序在同一台主机上启动多个独立 Worker，每个 Worker 拥有各自的浏览器实例，并从共享任务池中动态领取菜单任务或页级任务执行。程序并非简单地将任务平均分割后静态执行，而是通过共享队列、共享状态和页级任务分发，使多个 Worker 可以根据实时空闲情况动态协作。当某一任务执行速度较慢或其他窗口提前完成工作时，空闲 Worker 还可以加入现有任务的分页抓

取，从而提高整体资源利用率。此外，程序还支持断点续爬、失败任务重试和异常回收机制，使系统在遭遇网络波动、页面加载失败或人工中断时，仍能够依赖进度表恢复抓取过程。正因为如此，该方案虽然运行于单机环境，却具备了较强的伪分布式协同抓取特征。对应内容见图 2.39、图 2.40 和图 2.41。

```
1837 def worker_process(  
1838     worker_id: int,  
1839     task_list: list,  
1840     headless: bool,  
1841     wait_seconds: int,  
1842     window_rect: dict,  
1843     mysql_config: dict,  
1844     shared_done_detail_dict: dict,  
1845     shared_detail_lock,  
1846     shared_task_pool: list,  
1847     shared_task_pool_lock,  
1848     shared_page_task_pool: list,  
1849     shared_page_task_pool_lock,  
1850     shared_task_runtime: dict,  
1851     shared_task_runtime_lock,  
1852     driver_path: str  
1853 ):  
    pass
```

图 2.39 多进程 Worker 任务调度代码图

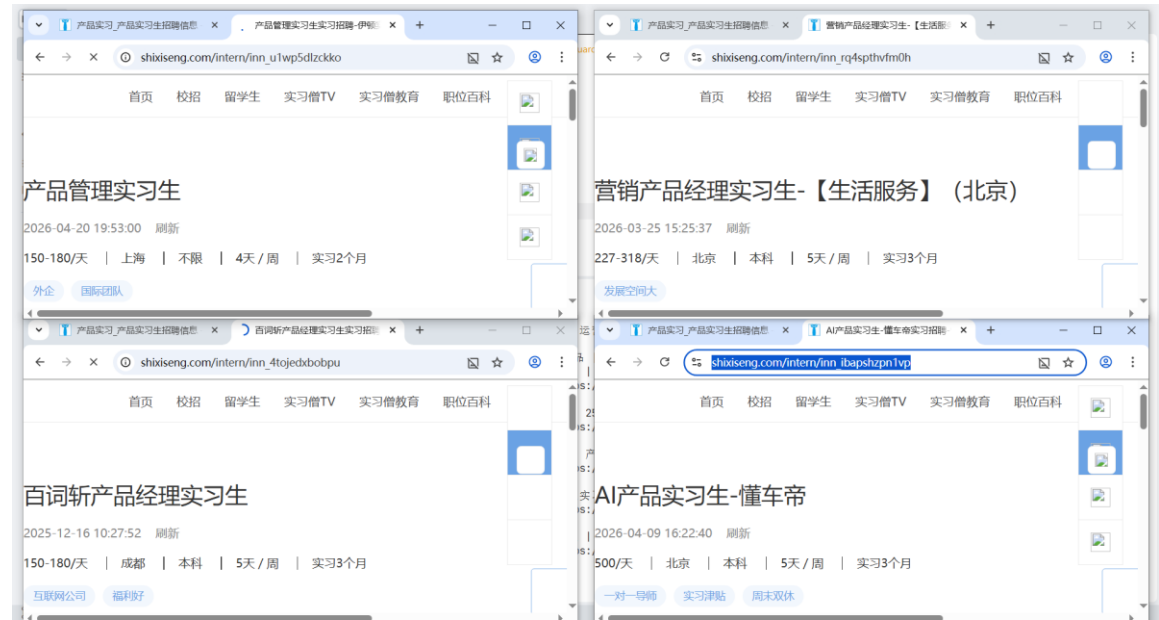


图 2.40 伪分布式浏览器协同运行图

```
○ (base) PS E:\Jupyter_Notebook\课程设计> & E:/anaconda3/envs/reptile/python.exe e:/Jupyter_Notebook/课程设计/shixiseng_optimized_empty_guard.py
[2026-04-22 23:31:49] 主进程启动。
[2026-04-22 23:31:49] 开始连接主 MySQL...
[2026-04-22 23:31:49] MySQL 已连接: shixiseng
[2026-04-22 23:31:50] MySQL 数据表已就绪。
[2026-04-22 23:31:50] 总任务数: 89
[2026-04-22 23:31:50] 待完成任务数: 89
[2026-04-22 23:31:50] 已存在详情联合键数: 11
[2026-04-22 23:31:50] MySQL 已关闭。
[2026-04-22 23:31:50] 主进程开始解析 ChromeDriver 路径...
[2026-04-22 23:31:50] ChromeDriver 路径解析成功: C:\Users\MSIPC\.wdm\drivers\chromedriver\win64\146.0.7680.153\chromedriver.exe
[2026-04-22 23:31:50] Worker-1 分配任务数: 23
[2026-04-22 23:31:50] Worker-2 分配任务数: 22
[2026-04-22 23:31:50] Worker-3 分配任务数: 22
[2026-04-22 23:31:50] Worker-4 分配任务数: 22
[2026-04-22 23:31:50] 准备启动 Worker-1 ...
[2026-04-22 23:31:50] Worker-1 已启动, PID=30636
[2026-04-22 23:31:51] 准备启动 Worker-2 ...
[2026-04-22 23:31:51] Worker-2 已启动, PID=28716
[2026-04-22 23:31:51] 准备启动 Worker-3 ...
[2026-04-22 23:31:51] Worker-3 已启动, PID=29672
[2026-04-22 23:31:51] 准备启动 Worker-4 ...
[2026-04-22 23:31:51] Worker-4 已启动, PID=22108
```

图 2.41 伪分布式任务分配示意图

综合来看，主抓取程序并不是一个单纯逐页访问网页的脚本，而是一个集任务读取、链接规范化、分页调度、详情解析、结果入库和进度恢复于一体的小型数据采集系统。它与前置菜单任务构建脚本共同构成了本课题“菜单任务生成—主程序批量抓取—数据库结构化沉淀”的完整实施链路。其核心特点在于：通过单机多进程模拟伪分布式抓取，通过数据库进度表实现任务记忆，通过双标签页机制提高页面抓取稳定性，从而使岗位采集过程具备较好的连续性、可恢复性与工程实用性。

3 数据预处理

3.1 数据预处理目标

在完成实习僧平台互联网 IT 类实习岗位数据采集后，原始岗位详情数据虽然已经写入数据库，但其字段组织形式仍然更偏向于抓取结果保存，尚不能完全满足后续统计分析的直接使用需求。一方面，部分字段主要服务于抓取过程中的任务管理、结果定位与状态记录，在后续分析中的作用相对有限；另一方面，部分与岗位特征密切相关的字段仍以文本形式存储，例如薪资区间、实习时长、每周到岗天数以及岗位标签等，如果不进一步处理，后续 SQL 提取与特征分析的难度都会明显增加。基于此，本课题在数据采集完成之后增加了数据预处理环节，对原始岗位详情数据进行筛选、转换与结构化整理，为后续分析提供更规范的数据基础。

3.2 原始数据读取

本课题的数据预处理工作主要由 `job_details_preprocessing.ipynb` 脚本完成。该脚本以数据库中的原始岗位详情表 `shixiseng_job_details` 为输入，利用 Python 中的 Pandas、NumPy、PyMySQL、SQLAlchemy、JSON 和正则表达式等工具，对原始数据进行批量清洗和字段重构，并最终生成处理后的数据表 `processed_job_details`。从脚本运行结果来看，共读取原始数据 172050 行、30 列，说明前期采集阶段已经积累了较大规模的岗位样本，也进一步体现出数据预处理在整个分析流程中的必要性。对应内容见图 3.1、图 3.2 和图 3.3。

```
[7]: 1 # 用于从 MySQL 读取原始数据并写入处理后的数据表
    2 DB_CONFIG = {
    3     "host": "127.0.0.1",
    4     "port": 3306,
    5     "user": "root",
    6     "password": "123456",
    7     "database": "shixiseng_db",
    8     "charset": "utf8" # 数据库字符集, utf8_general_ci 排序规则
    9 }
   10
   11 SOURCE_TABLE = "shixiseng_job_details"
   12 PROCESSED_TABLE = "processed_job_details"
   13
   14 # 创建 SQLAlchemy 引擎, 用于数据读写
   15 engine = create_engine(
   16     f"mysql+pymysql://{DB_CONFIG['user']}:{DB_CONFIG['password']}@"
   17     f"{DB_CONFIG['host']}:{DB_CONFIG['port']}/{DB_CONFIG['database']}?charset=utf8"
   18 )
```

图 3.1 MySQL 数据库连接代码图

```
[9]: 1 # 将原始招聘数据表加载到 DataFrame, 便于后续分析和处理
2 df = pd.read_sql(f"SELECT * FROM {SOURCE_TABLE}", engine)
3
4 print(f"原始数据行数: {len(df)}, 列数: {df.shape[1]}")
5 print(df.head())
```

图 3.2 原始岗位数据读取代码图

```
原始数据行数: 172050, 列数: 30
id group_name item_text
item_url search_page intern_id \
0 1 产品 产品助理 https://www.shixiseng.com/interns?keyword=%E4%B%A7%E5%93%81%E5%8A%A9%E7%90%86&city=%E5%85%A8%E5%9
B%BD&type=intern&from=menu 1 inn_83amnc5w13bs
1 2 产品 产品 https://www.shixiseng.com/interns?keyword=%E4%B%A7%E5%93%81&city=%E5%85%A8%E5%9B%
BD&type=intern&from=menu 1 inn_hpywzaipk8ss
2 3 产品 产品经理 https://www.shixiseng.com/interns?keyword=%E4%B%A7%E5%93%81%E7%BB%8F%E7%90%86&city=%E5%85%A8%E5%9
B%BD&type=intern&from=menu 1 inn_hpywzaipk8ss
3 4 产品 产品设计 https://www.shixiseng.com/interns?keyword=%E4%B%A7%E5%93%81%E8%AE%BE%E8%AE%A1&city=%E5%85%A8%E5%9
B%BD&type=intern&from=menu 1 inn_g0jz3q6y8
4 5 产品 产品助理 https://www.shixiseng.com/interns?keyword=%E4%B%A7%E5%93%81%E5%8A%A9%E7%90%86&city=%E5%85%A8%E5%9
B%BD&type=intern&from=menu 1 inn_7q7frqeyfbgs

detail_url job_title_detail publish_time_detail salary_detail city_deta
il degree_detail days_per_week_detail intern_months_detail \
0 https://www.shixiseng.com/intern/inn_83amnc5w13bs?pcm=pc_SearchList 产品助理 2026-01-06 22:29:06 150-300/天
南京 本科 4天/周 实习3个月
1 https://www.shixiseng.com/intern/inn_hpywzaipk8ss?pcm=pc_SearchList 产品经理 2026-03-18 17:30:13 薪资面议
上海 本科 5天/周 实习3个月
2 https://www.shixiseng.com/intern/inn_hpywzaipk8ss?pcm=pc_SearchList 产品经理 2026-03-18 17:30:13 薪资面议
上海 本科 5天/周 实习3个月
3 https://www.shixiseng.com/intern/inn_g0jz3q6y8?pcm=pc_SearchList 产品设计实习生 2026-03-20 14:14:11 120-150/天
北京 本科 5天/周 实习3个月
4 https://www.shixiseng.com/intern/inn_7q7frqeyfbgs?pcm=pc_SearchList 产品助理实习生 2026-03-09 11:20:00 120-140/天
广州 本科 4天/周 实习3个月

job_tags_detail job_header_img_links \
0 可转正实习、接受一大二、零基础实习、提供免费住宿、一对一导师、免费班车、周末双休
1 周末双休 https://sxsimg.xiaoyuanzhao.com/static/new_main/img/ji.svg
2 周末双休 https://sxsimg.xiaoyuanzhao.com/static/new_main/img/ji.svg
3 学习机会多、成长空间大、免费三餐
4 地铁周边、周末双休、一对一导师、留学生实习、节日福利、产品开发

job_description \
0 岗位职责 1、负责产品线的需求分析, 逻辑梳理, 策略制定, 设计规划和上线迭代以及相关运营工作; 2、基于对用户模型、动机事件分析、业务分
析, 进行需求架构; 3、与市场、设计、技术等团队紧密配合, 推进产品开发测试; 4、产品上线后积极跟踪, 进行效果评估, 制定迭代优化方案。 任职
资格 1、985-211院校本科及以上学历; 2、具备优秀的学习执行能力、数据分析能力、团队合作意识; 3、具备较强的...
1 招聘对象: 海内外2027届应届毕业生(毕业时间: 2026.11-2027.10) *中国大陆(内地)以毕业证为准, 中国港澳台及海外地区以学位证为准 工作地
点: 上海 职位描述: 1. 深入理解业务场景, 通过用户调研、行为分析及研究, 挖掘用户真实痛点及需求, 定义出核心产品价值; 2. 负责产品功能的规
划及原型设计, 产出高质量需求文档(PRD), 推动产品从“概念”走向“落地”; 3. 协同研发、...
2 招聘对象: 海内外2027届应届毕业生(毕业时间: 2026.11-2027.10) *中国大陆(内地)以毕业证为准, 中国港澳台及海外地区以学位证为准 工作地
点: 上海 职位描述: 1. 深入理解业务场景, 通过用户调研、行为分析及研究, 挖掘用户真实痛点及需求, 定义出核心产品价值; 2. 负责产品功能的规
划及原型设计, 产出高质量需求文档(PRD), 推动产品从“概念”走向“落地”; 3. 协同研发、...
3 岗位职责: 1. 协助设计团队完成App及Web端的日常产品界面设计工作, 包括但不限于活动设计、中小需求跟进、方案延展适配等。 2. 根据设计规范和
具体需求高效高质量输出设计方案; 理解业务逻辑, 与研发工程师配合, 确保设计落地还原。 3. 在开放性项目中能提出自己的创意思考, 参与设计风格的研究
和探索。 4. 关注行业内设计趋势, 为团队提供设计灵感和新鲜视角。 任职要求: 1. 能稳定一周至少4天...
4 岗位职责 ● 协助产品负责人进行产品功能设计, 完成市场调研与用户需求分析, 撰写结构清晰、要素完整的需求文档(含背景说明、功能逻辑、原型图
及埋点规划); ● 与技术、运营等相关部门配合沟通, 提供相关支持; ● 支持团队的日常运营, 包括数据整理和报告准备等。 任职要求 ● 本科及以上学历,
欢迎大三、大四在校生投递, 实习期需保证连续3个月, 每周4天; ● 英语四级以上, 能进行英文书面及口语沟通者...

resume_language deadline_detail job_address company_name_detail company_desc_detail
company_tags_detail industry_detail company_nature_detail \
0 中文 2026-10-24 联东U谷滨水科技产业园 丁丁停车 “专注车
位的共享经济” 智能硬件、新能源、高科技 互联网/游戏/软件 民营企业
1 中文 2026-06-23 上海市/上海市/普陀区 淘宝闪购 “自2025年12月起, “饿了么”品
牌更名为“淘宝闪购”。” 年轻人、外卖行业头牌、互联网新零售、高速增长、奋斗沃土 互联网/游戏/软件 民营企业
2 中文 2026-06-23 上海市/上海市/普陀区 淘宝闪购 “自2025年12月起, “饿了么”品
牌更名为“淘宝闪购”。” 年轻人、外卖行业头牌、互联网新零售、高速增长、奋斗沃土 互联网/游戏/软件 民营企业
3 中文 2026-05-30 学清路甲 18 号中国产学研成果转化基地西一门 知乎 “让人们更好的分享
知识和见解, 找到自己的解答。” 行业口碑、办公用品、不时时聚餐、大咖、公司氛围好、工作环境舒适 互联网/游戏/软件
民营企业
4 中文 2026-09-05 广东省/广州市/天河区 广晟国际大厦 ChatLabs “ChatLabs创立于2016年, 是领先数字化、
自动化、智能化营销服务供应商。” 广告营销、氛围轻松、年轻人、优势资源、外企氛围、学习机会多 互联网/游戏/软件
民营企业

company_size_detail company_location_detail company_intro_img_links created_at updated_at
0 50-150人 江苏 2026-03-22 00:33:41 2026-03-22 00:33:41
1 2000人以上 上海市/上海 2026-03-22 00:33:42 2026-03-22 00:33:42
2 2000人以上 上海市/上海 2026-03-22 00:33:42 2026-03-22 00:33:42
3 2000人以上 北京市 2026-03-22 00:33:42 2026-03-22 00:33:42
4 50-150人 广东/广州 2026-03-22 00:33:57 2026-03-22 00:33:57
```

图 3.3 原始岗位数据基本信息图

3.3 删除无关列与冗余字段

在原始数据读取完成后，预处理脚本首先结合后续分析任务的实际需求，对字段进行适当精简。像 `id`、`intern_id`、`item_url`、`detail_url`、`search_page`、`created_at`、`updated_at` 等字段，主要用于抓取阶段的任务管理、过程记录和结果定位，在后续岗位分布、薪资特征和技能要求分析中的直接使用价值相对较低，因此在预处理阶段予以删除。与此同时，一些篇幅较长但当前分析中并不直接使用的字段，如岗位原始描述、公司介绍和图片链接等，也进行了删减处理。经过这一环节后，处理后的数据表在字段组织上更加紧凑，能够更集中地服务于后续分析主题。对应内容见图 3.4。

```
[11]: 1 # 删除前的列名
2 print("删除前的列名:")
3 print(df.columns.tolist())
4
5 # 去掉分析中不需要的列, 减少冗余数据
6 # 包括 URL、ID、时间戳、文本描述等, 避免存储和计算冗余
7 columns_to_drop = [
8     'item_url', 'id', 'search_page', 'intern_id', 'detail_url',
9     'resume_language', 'job_address', 'job_description',
10    'company_desc_detail', 'company_tags_detail',
11    'company_intro_img_links', 'created_at', 'updated_at'
12 ]
13 df.drop(columns=columns_to_drop, inplace=True)
14
15 # 删除后的列名
16 print("删除后的列名:")
17 print(df.columns.tolist())

删除前的列名:
['id', 'group_name', 'item_text', 'item_url', 'search_page', 'intern_id', 'detail_url', 'job_title_detail', 'publish_time_detail',
'salary_detail', 'city_detail', 'degree_detail', 'days_per_week_detail', 'intern_months_detail', 'job_tags_detail', 'job_header_img_
links', 'job_description', 'resume_language', 'deadline_detail', 'job_address', 'company_name_detail', 'company_desc_detail', 'compa
ny_tags_detail', 'industry_detail', 'company_nature_detail', 'company_size_detail', 'company_location_detail', 'company_intro_img_li
nks', 'created_at', 'updated_at']
删除后的列名:
['group_name', 'item_text', 'job_title_detail', 'publish_time_detail', 'salary_detail', 'city_detail', 'degree_detail', 'days_per_we
ek_detail', 'intern_months_detail', 'job_tags_detail', 'job_header_img_links', 'deadline_detail', 'company_name_detail', 'industry_d
etail', 'company_nature_detail', 'company_size_detail', 'company_location_detail']
```

图 3.4 删除冗余字段前后列名对比图

3.4 关键特征变量构造

为便于后续 SQL 提取和可视化分析，预处理阶段在删除冗余字段的基础上，继续围绕岗位薪资、岗位标签、实习要求和急招状态等内容构造关键特征变量。原始岗位详情数据虽然已经保存了较完整的网页信息，但其中部分字段仍以文本、链接或混合字符串的形式存在，难以直接用于分组统计、条件筛选和图表展示。例如，岗位是否急招原本隐含在页面图片链接中，岗位标签通常以普通字符串形式保存，薪资字段中既包含区间薪资，也包含“薪资面议”等非数值内容，实习时长和每周到岗天数也需要从文本中进一步提取数字信息。因此，有必要在预处理阶段将这些

字段统一转换为结构化变量，为后续高薪岗位筛选、急招岗位统计、岗位标签分析和实习要求比较提供稳定的数据基础。关键特征变量构造情况如表 3.1 所示。

表 3.1 关键特征变量构造说明表

新字段	来源字段	处理方法	字段含义
is_urgent_hire	job_header_img_links	判断是否包含急招标识	标识岗位是否为急招岗位
job_tags	job_tags_detail	拆分岗位标签并转换为 JSON 字符串	保存结构化岗位标签
is_salary_negotiable	salary_detail	判断是否为薪资面议或缺失	标识岗位薪资是否可计算
salary_min_per_day	salary_detail	使用正则表达式提取最低日薪	表示岗位最低日薪
salary_max_per_day	salary_detail	使用正则表达式提取最高日薪	表示岗位最高日薪
internship_months	intern_months_detail	提取实习时长中的月份数字	表示实习月数
days_per_week	days_per_week_detail	提取每周到岗天数字	表示每周到岗天数

3.4.1 “急招”标志变量构造

在字段清理的基础上，预处理脚本首先对急招信息进行结构化处理。原始数据中岗位是否属于“急招”并没有以独立字段给出，而是隐含在页面头图链接信息中。为便于后续统计，脚本根据 job_header_img_links 中是否包含急招标识，构造新的二元变量 is_urgent_hire，以 1 表示急招岗位，以 0 表示非急招岗位。通过这一处理，网页中的视觉提示信息被转换为能够直接参与统计分析的结构化字段，从而为后续比较急招岗位与普通岗位在薪资、城市和岗位方向上的差异提供了条件。对应内容见图 3.5。

```
[13]: 1 # 先生成急招标志列
2 df['is_urgent_hire'] = df['job_header_img_links'].fillna('').str.contains(
3     "https://xsimg.xiaoyuanzhao.com/static/new_main/img/ji.svg"
4 ).astype(int)
5
6 print("急招标志生成结果预览：")
7 print(df[['job_header_img_links', 'is_urgent_hire']].head(10))

急招标志生成结果预览：
```

	job_header_img_links	is_urgent_hire
0		0
1	https://xsimg.xiaoyuanzhao.com/static/new_main/img/ji.svg	1
2	https://xsimg.xiaoyuanzhao.com/static/new_main/img/ji.svg	1
3		0
4		0
5	https://xsimg.xiaoyuanzhao.com/static/new_main/img/ji.svg	1
6	https://xsimg.xiaoyuanzhao.com/static/new_main/img/ji.svg	1
7		0
8		0
9		0

```
[14]: 1 # 确认无误后再删除原列
2 df.drop(columns='job_header_img_links', inplace=True)
```

图 3.5 “急招”标志变量构造图

3.4.2 岗位标签结构化处理

岗位标签字段在原始数据中通常以字符串形式保存，不同标签之间使用顿号分隔。为了兼顾数据库存储和后续程序调用的便利性，预处理脚本先对 `job_tags_detail` 字段进行拆分，再将拆分结果转换为 JSON 字符串，生成新的 `job_tags` 字段，最后删除原始字段 `job_tags_detail`。经过这一处理后，原本以简单文本形式保存的岗位标签被转换为更具结构化特征的数据形式，既保留了原始标签信息，也提高了字段在后续标签统计、技能归纳与岗位画像分析中的可用性。对应内容见图 3.6。

```
[17]: 1 # 拆分成列表
      2 df['job_tags'] = df['job_tags_detail'].fillna('').str.split(',')

[18]: 1 print("拆分为列表后的 job_tags 前5行: ")
      2 print(df['job_tags'].head())

拆分为列表后的 job_tags 前5行:
0    [可转正实习, 接受大大二, 零基础实习, 提供免费住宿, 一对一导师, 免费班车, 周末双休]
1                                           [周末双休]
2                                           [周末双休]
3    [学习机会多, 成长空间大, 免费三餐]
4    [地铁周边, 周末双休, 一对一导师, 留学生实习, 节日福利, 产品开发]
Name: job_tags, dtype: object

[19]: 1 df['job_tags'] = df['job_tags'].apply(lambda x: json.dumps(x, ensure_ascii=False)) # JSON 存储并保留中文

[20]: 1 print("转为 JSON 后的 job_tags 前5行: ")
      2 print(df['job_tags'].head())

转为 JSON 后的 job_tags 前5行:
0    ["可转正实习", "接受大大二", "零基础实习", "提供免费住宿", "一对一导师", "免费班车", "周末双休"]
1                                           ["周末双休"]
2                                           ["周末双休"]
3    ["学习机会多", "成长空间大", "免费三餐"]
4    ["地铁周边", "周末双休", "一对一导师", "留学生实习", "节日福利", "产品开发"]
Name: job_tags, dtype: object

[21]: 1 # 删除原始列
      2 df.drop(columns='job_tags_detail', inplace=True)
```

图 3.6 岗位标签字段结构化处理图

3.4.3 薪资字段数值化处理

薪资字段的处理是本次数据预处理中的重点环节之一。原始表中的 `salary_detail` 多以文本形式存在，其中既包括区间型日薪和单值型薪资，也包括“薪资面议”等非数值描述。若直接保留原始文本形式，不利于后续识别高薪岗位特征。因此，预处理脚本首先构造 `is_salary_negotiable` 字段，用于标识岗位是否属于薪资面议或缺失状态；随后对薪资文本中的单位信息进行清理，并利用正则表达式提取薪资区间中的最小值与最大值，最终生成 `salary_min_per_day` 和 `salary_max_per_day` 两个数值字段。通过这一转换，原本难以直接比较的文本薪资信息被转化为可排序、可筛选、可统计的结构化数值变量，为后续高薪岗位分析提供了基础。对应内容见图 3.7。

```
[24]: 1 salary_col = df['salary_detail'].fillna('')
      2
      3 # is_salary_negotiable: 是否薪资面议, 1 表示面议, 0 表示固定薪资
      4 df['is_salary_negotiable'] = salary_col.isin(['薪资面议', '', 'nan', 'None']).astype(int)
      5
      6 # 向量化提取数字区间, 提高处理效率
      7 salary_clean = salary_col.str.replace('/天', '', regex=False)
      8 range_match = salary_clean.str.extract(r'(\d+)-(\d+)').astype(float)
      9 single_match = salary_clean.str.extract(r'^(\d+)$').astype(float)
      10
      11 # salary_min_per_day / salary_max_per_day: 日薪最小值和最大值
      12 df['salary_min_per_day'] = range_match[0].combine_first(single_match[0])
      13 df['salary_max_per_day'] = range_match[1].combine_first(single_match[0])

[25]: 1 print("薪资字段处理结果前10行: ")
      2 df[['salary_detail', 'is_salary_negotiable', 'salary_min_per_day', 'salary_max_per_day']].head(10)

薪资字段处理结果前10行:
```

	salary_detail	is_salary_negotiable	salary_min_per_day	salary_max_per_day
0	150-300/天	0	150.0	300.0
1	薪资面议	1	NaN	NaN
2	薪资面议	1	NaN	NaN
3	120-150/天	0	120.0	150.0
4	120-140/天	0	120.0	140.0
5	薪资面议	1	NaN	NaN
6	薪资面议	1	NaN	NaN
7	150-180/天	0	150.0	180.0
8	100/天	0	100.0	100.0
9	100-200/天	0	100.0	200.0

```
[26]: 1 df.drop(columns='salary_detail', inplace=True)
```

图 3.7 薪资字段数值化处理图

3.4.4 实习时长与每周工作天数字段提取

除薪资字段外，预处理脚本还对岗位要求中的时间类文本字段进行了进一步结构化处理。例如，intern_months_detail 原本以“实习若干个月”的文字形式记录实习周期，days_per_week_detail 则以“每周若干天”的方式表示到岗要求。预处理阶段通过正则表达式分别提取这些字段中的数字信息，并生成 internship_months 和 days_per_week 两个数值字段。通过这一处理，原本只能以文本方式阅读的岗位要求被转换为可直接参与统计分析的定量变量，使实习周期和到岗强度能够在 SQL 查询和后续可视化分析中被统一调用。这样不仅有利于分析不同岗位在实习时长和每周到岗天数方面的分布特征，也能够为后续高薪岗位筛选、岗位要求比较和实习强度判断提供数据基础。对应内容见图 3.8 和图 3.9。

```
[29]: 1 # 提取实习时长 (月数)
      2 df['internship_months'] = df['intern_months_detail'].str.extract(r'(\d+)').astype(int)
```

```
[30]: 1 print("实习时长提取结果前10行: ")
      2 df[['intern_months_detail', 'internship_months']].head(10)
```

实习时长提取结果前10行:

```
[30]:
```

	intern_months_detail	internship_months
0	实习3个月	3
1	实习3个月	3
2	实习3个月	3
3	实习3个月	3
4	实习3个月	3
5	实习3个月	3
6	实习3个月	3
7	实习6个月	6
8	实习3个月	3
9	实习3个月	3

```
[31]: 1 df.drop(columns='intern_months_detail', inplace=True)
```

图 3.8 实习时长字段数值提取图

```
[34]: 1 # 提取每周工作天数
      2 df['days_per_week'] = df['days_per_week_detail'].fillna('0').str.extract(r'(\d+)').astype(int)
```

```
[35]: 1 print("每周工作天数提取结果前10行: ")
      2 print(df[['days_per_week_detail', 'days_per_week']].head(10))
```

每周工作天数提取结果前10行:

```
days_per_week_detail  days_per_week
```

0	4天/周	4
1	5天/周	5
2	5天/周	5
3	5天/周	5
4	4天/周	4
5	5天/周	5
6	5天/周	5
7	5天/周	5
8	5天/周	5
9	5天/周	5

```
[36]: 1 df.drop(columns='days_per_week_detail', inplace=True)
```

图 3.9 每周到岗天数字段数值提取图

3.5 预处理结果写入数据库

在完成字段删减、变量构造与格式转换后，预处理脚本将处理后的结果统一写入新的数据表 `processed_job_details`。该表并不是对原始岗位详情表 `shixiseng_job_details` 的简单复制，而是在保留核心岗位信息的基础上，进一步删除了与分析关系较弱的冗余字段，并新增了多个可直接参与统计分析的结构化变量。

从数据用途来看，原始表 `shixiseng_job_details` 更偏向于采集阶段的结果沉淀，主要用于保留从网页详情页中直接获取的岗位信息；而 `processed_job_details` 更偏向于分析阶段的数据基础，字段设置更加紧凑，数据类型更加规范，能够直接支撑后续 SQL 查询、岗位数量统计、薪资筛选、技能关键词匹配和可视化分析。因此，将预处理结果单独写入新表，有利于区分“原始采集数据”和“分析处理数据”，也便于后续在不破坏原始数据的前提下开展多轮分析。

在写入数据库时，脚本通过 `SQLAlchemy` 建立与 `MySQL` 的连接，并将 `DataFrame` 数据整体写入 `processed_job_details` 表。写入过程中，程序对部分字段设置了适合 `MySQL` 存储的字段类型，例如岗位标签字段使用文本类型保存，薪资、实习月数和每周到岗天数等字段则以数值型变量参与后续分析。这样既保证了数据能够顺利入库，也提高了后续查询和统计处理的便利性。预处理结果写入数据库的代码如图 3.10 所示。

```
[40]: v 1 dtype_mapping = {
2       'job_tags': sqltypes.Text(collation='utf8_general_ci'),
3       'job_title_detail': sqltypes.VARCHAR(255, collation='utf8_general_ci'),
4       'item_text': sqltypes.VARCHAR(255, collation='utf8_general_ci'),
5       'company_name_detail': sqltypes.VARCHAR(255, collation='utf8_general_ci')
6   }
7
8   df.to_sql(
9       PROCESSED_TABLE,
10      engine,
11      if_exists='replace',
12      index=False,
13      dtype=dtype_mapping
14  )
15
16  print(f"处理后的数据已成功写入数据库表: {PROCESSED_TABLE}")
```

处理后的数据已成功写入数据库表: `processed_job_details`

图 3.10 预处理结果写入数据库代码图

通过该步骤，数据预处理阶段形成了较为完整的输出结果：一方面，原始岗位详情表继续保留网页采集阶段的完整信息，便于后续追溯和核查；另一方面，预处理结果表则作为后续数据提取和可视化分析的直接数据来源，承担分析型数据表的作用。由此，本课题完成了从原始网页数据到结构化分析数据的转换，为第 4 章数据库存储说明和第 5 章数据提取分析奠定了基础。

4 数据存储

4.1 选择数据存储方式

本课题选择 MySQL 作为数据存储工具。其主要原因在于，互联网 IT 类实习岗位数据具有较明确的字段结构，适合采用关系型数据库进行统一管理。同时，MySQL 支持主键、唯一键、索引和事务控制，便于实现任务去重、状态维护、结果写入与后续查询分析。此外，本课题后续的数据提取主要依赖 SQL 语句完成，因此使用 MySQL 作为核心存储平台，不仅能够满足抓取阶段的结果沉淀需求，也能够为预处理结果管理和后续统计分析提供良好的支撑。

与仅将数据保存为 Excel 或 CSV 文件相比，MySQL 在本课题中的优势更加明显。一方面，数据库能够同时承担任务管理、抓取进度记录和结果存储等多种功能；另一方面，在数据量较大的情况下，数据库更适合支持多表组织、条件筛选和聚合统计。因此，从系统完整性和后续分析便利性来看，MySQL 更符合本课题从数据采集、数据预处理到数据提取分析全过程一体化实现的要求。

4.2 构建数据存储结构

围绕实习僧平台互联网 IT 类实习岗位的数据采集与分析需求，本课题构建了由输入任务表、原始详情表、预处理结果表以及进度表共同组成的数据存储结构。该结构并不是简单保存抓取结果，而是同时承担任务入口管理、抓取结果沉淀、预处理结果输出、任务状态记忆和后续分析支撑等多重职责，从而形成较为完整的数据管理链路。

其中，shixiseng_internet_it_links 表由前置任务构建脚本写入，用于保存从实习僧首页“互联网 IT”菜单中提取出的岗位分类任务；shixiseng_job_details 表由主抓取程序写入，用于保存从列表页与详情页提取出的原始岗位详情信息；processed_job_details 表由预处理脚本生成，用于保存删除冗余字段、完成变量构造和数值化处理后的分析型数据；shixiseng_task_progress 和 shixiseng_detail_progress 两张表则分别记录菜单任务页级进度和详情页完成状态，是实现断点续爬、进度记忆和多进程协同抓取的重要支撑。各数据表在系统中的层级和作用如表 4.1 所示。

表 4.1 数据库存储表功能说明表

数据表	所属层级	主要内容	作用
shixiseng_internet_it_links	任务入口层	岗位分组、岗位名称、搜索链接	为主抓取程序提供标准化任务源
shixiseng_job_details	原始结果层	原始岗位标题、城市、薪资、标签、公司信息等	保存网页抓取得到的原始岗位详情
processed_job_details	分析准备层	清洗后的岗位字段和新增分析变量	支撑 SQL 提取和可视化分析
shixiseng_task_progress	任务进度层	每个菜单任务的页级抓取进度	支持任务断点续爬
shixiseng_detail_progress	详情进度层	已完成抓取的详情页记录	防止同一详情页重复抓取

从结构关系上看，输入任务表位于整个系统的数据入口层，负责提供标准化抓取任务；原始详情表位于采集结果沉淀层，负责保存网页抓取所得的完整岗位数据；预处理结果表位于分析准备层，负责将原始详情数据进一步整理为适合统计分析的结构化结果；两张进度表则位于状态控制层，保障抓取任务能够在中断后恢复执行。通过这样的分层设计，本课题的数据存储体系能够较好地支撑从任务生成到结果分析的完整流程。

为了更直观地展示各数据表之间的组织关系，本课题对数据库存储结构进行了整理，如图 4.1 所示。从图中可以看出，输入任务表、原始岗位详情表、预处理结果表和进度表共同构成了完整的数据存储体系，其中任务表提供抓取入口，详情表保存原始结果，预处理表支撑后续分析，进度表用于维护抓取状态。

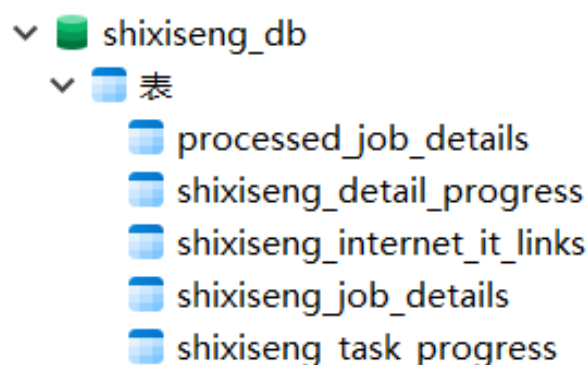


图 4.1 数据库存储结构图

在具体表结构方面，原始岗位详情表是采集阶段最核心的数据结果表，其字段设计直接决定后续数据预处理和分析工作的基础。该表结构如图 4.2 所示，其中既包含岗位标题、城市、薪资、实习要求、岗位标签、公司信息等业务字段，也包含详情链接、任务来源和入库时间等辅助字段，能够较完整地保存岗位详情页中的主要信息。

字段	索引	外键	检查	触发器	选项	注释	SQL 预览					
名						类型	长度	小数点	不是 null	虚拟	键	
▶ id						bigint	20		☒	☐		1
group_name						varchar	255		☒	☐		
item_text						varchar	255		☒	☐		
item_url						text			☒	☐		
search_page						int	11		☐	☐		
intern_id						varchar	100		☐	☐		
detail_url						text			☒	☐		
job_title_detail						varchar	255		☐	☐		
publish_time_detail						varchar	255		☐	☐		
salary_detail						varchar	255		☐	☐		
city_detail						varchar	255		☐	☐		
degree_detail						varchar	255		☐	☐		
days_per_week_detail						varchar	255		☐	☐		
intern_months_detail						varchar	255		☐	☐		
job_tags_detail						text			☐	☐		
job_header_img_links						text			☐	☐		
job_description						longtext			☐	☐		
resume_language						varchar	255		☐	☐		
deadline_detail						varchar	255		☐	☐		
job_address						text			☐	☐		
company_name_detail						varchar	255		☐	☐		
company_desc_detail						text			☐	☐		
company_tags_detail						text			☐	☐		
industry_detail						varchar	255		☐	☐		
company_nature_detail						varchar	255		☐	☐		
company_size_detail						varchar	255		☐	☐		
company_location_detail						varchar	255		☐	☐		
company_intro_img_links						text			☐	☐		
created_at						timestamp			☐	☐		
▶ updated_at						timestamp			☐	☐		

图 4.2 原始岗位详情表结构图

4.3 展示数据存储结果

在数据写入过程中，系统首先由前置脚本完成菜单分类任务入库，形成输入任务表；随后主抓取程序持续写入岗位详情数据和抓取进度信息，形成原始结果表与进度表；最后由预处理脚本对原始详情数据进行字段清理、变量构造和结构化转换，并生成处理后的结果表。由此，本课题最终形成了以任务表驱动抓取、以原始表沉淀结果、以预处理表支撑分析、以进度表维护状态的一体化数据存储体系。

从输入任务表来看，其主要保存由实习僧首页“互联网 IT”菜单转换而来的岗位分类任务记录，每条任务均对应具体的岗位名称和标准化搜索链接。输入任务表示例数据如图 4.3 所示。该表虽然不直接参与后续统计分析，但它为伪分布式抓取程序提供了统一、可重复调度的数据入口，是整个采集系统稳定运行的基础。

对象 shixiseng_internet_it_links @shixisen...					
开始事务 文本 筛选 排序 导入 导出 数据生成 创建图表					
id	group_name	item_text	item_url	created_at	updated_at
1	产品	产品经理	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
2	产品	产品助理	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
3	产品	产品设计	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
4	产品	产品项目管理	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
5	产品	用户研究	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
6	运营	运营	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
7	运营	运营助理	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
8	运营	运营专员	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
9	运营	产品运营	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
10	运营	内容运营	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
11	运营	SEO/SEM运营	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
12	运营	电商运营	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
13	运营	活动运营	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
14	运营	数据运营	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
15	运营	社群运营	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
16	运营	新媒体运营	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
17	运营	游戏运营	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
18	运营	海外运营	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
19	运营	品类运营	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
20	运营	用户运营	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
21	运营	网站运营	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
22	运营	文案编辑	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
23	技术管理	技术管理	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
24	技术管理	架构师	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
25	后端开发	后端开发	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
26	后端开发	C/C++	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
27	后端开发	Java	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
28	后端开发	ERP技术开发	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
29	后端开发	PHP	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
30	后端开发	Python	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
31	后端开发	Ruby	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
32	后端开发	Linux	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
33	后端开发	Hadoop	https://www.shixiseng.co	2026-03-21 03:03:52	2026-04-08 01:09:56
34	后端开发				

图 4.3 输入任务表示例数据图

原始岗位详情表主要保存主抓取程序直接写入的岗位详情结果，字段内容相对完整。原始岗位详情表示例数据如图 4.4 所示。该表既包括岗位标题、薪资、城市、学历要求、实习时长、职位描述、公司名称等业务信息，也保留了部分与抓取过程相关的辅助字段，因此更适合作为采集阶段的原始数据沉淀表。

对象 shixiseng_job_details @shixisen...							
提交 回收 文本 筛选 排序 导入 导出 数据生成 创建图表							
degree_detail	days_per_week_detail	intern_months_detail	job_tags_detail	job_header_img_links	job_description	resume_language	deadline_detail
本科	4天/周	实习3个月	可转正实习、接受大一、大二、		岗位职责：1、负责产品线的...	2026-10-24	
本科	5天/周	实习3个月	周末双休	https://xsimg.xiaoyuanz...	招聘对象：海内外2027届...	2026-06-23	
本科	5天/周	实习3个月	周末双休	https://xsimg.xiaoyuanz...	招聘对象：海内外2027届...	2026-06-23	
本科	5天/周	实习3个月	学习机会多、成长空间大、...		岗位职责：1、协助设计团队...	2026-05-30	
本科	4天/周	实习3个月	地铁周边、周末双休、一对...		岗位职责：1、协助产品负责人...	2026-09-05	
本科	5天/周	实习3个月	周末双休	https://xsimg.xiaoyuanz...	招聘对象：海内外2027届...	2027-03-18	
本科	5天/周	实习3个月	周末双休	https://xsimg.xiaoyuanz...	招聘对象：海内外2027届...	2027-03-18	
本科	5天/周	实习6个月	地铁周边、零基础实习、一...		a、辅助设计师完成产品资料...	2026-08-28	
大专	5天/周	实习3个月	暑期实习、零基础实习		岗位职责：1、产品运营；2、...	2026-03-31	
本科	5天/周	实习3个月	不加班、远程实习、节日福利		1、根据公司业务发展需要，协...	2026-03-31	
本科	5天/周	实习6个月	互联网公司、福利好		岗位职责：1、常规定期用...	2026-12-21	
本科	5天/周	实习6个月	实习证明、晚餐补贴、免费...		岗位职责：(具体工作内容)...	2026-12-30	
本科	5天/周	实习6个月	AI产品、1v1带教、团队氛围...		(我们期待与你一起探索 AI...	2026-09-29	
本科	4天/周	实习3个月	产品好、领导nice		工作职责：1、调研、分析...	2026-03-31	
本科	4天/周	实习4个月	团队氛围好、晋升空间大、...		职位描述：日常实习：面向全...	2026-04-12	
本科	5天/周	实习3个月	可转正实习		岗位职责：1、竞品分析；2、...	2026-03-31	
本科	5天/周	实习3个月	定期团建		工作职责：1、负责海外SaaS...	2026-03-31	
本科	5天/周	实习3个月	餐补、周末双休		职位描述：负责得物交易与社...	2026-11-08	
本科	4天/周	实习4个月	团队氛围好、弹性工作、租...		职位描述：日常实习：面向全...	2026-04-12	
本科	4天/周	实习6个月	接受大一、大二、留学生实习、		【工作内容】：1、Zennu中...	2026-03-31	
本科	5天/周	实习3个月	一对一带教		工作职责：参与百度文库API...	2027-01-01	
本科	5天/周	实习3个月	弹性工作、免费三餐、扁平...		工作职责：1、深度调研客户...	2026-10-10	
本科	4天/周	实习4个月	团队氛围好、晋升空间大、...		职位描述：日常实习：面向全...	2026-04-12	
硕士	5天/周	实习3个月	暑期实习		一、工作职责：1、主要负责...	2026-03-31	
本科	5天/周	实习3个月	餐补、地铁周边、周末双休、		岗位职责：1、参与AI辅助创作...	2026-03-27	
本科	5天/周	实习3个月	餐补、地铁周边、周末双休、		岗位职责：1、参与AI辅助创作...	2026-03-27	
本科	5天/周	实习6个月	地铁周边、周末双休		岗位职责：1、对各友商OS及...	2026-03-25	
不限	4天/周	实习4个月	初创、腾讯背景创始人、互...		岗位职责：1、负责对市场不...	2026-03-31	
本科	4天/周	实习3个月	团队氛围好、弹性工作、租...		职位描述：日常实习：面向全...	2026-04-12	
本科	5天/周	实习6个月	留学生实习、零基础实习、...		职位描述：1、通过新形态中...	2026-04-14	
本科	5天/周	实习3个月	免费班车、免费晚餐、下午...		【岗位定位】这是一个兼具...	2026-06-30	
本科	5天/周	实习6个月	可转正实习、双导师培养、...		岗位职责：1、协助产品经...	2026-03-31	
不限	5天/周	实习3个月	可转正实习、定期团建、不...		职位描述：1、负责产品...	2026-03-31	
本科	5天/周	实习9个月	免费班车、氛围好、环境舒...		岗位内容：1、负责人才...	2026-03-31	

图 4.4 原始岗位详情表示例数据图

预处理结果表是在原始岗位详情表基础上进一步清洗和转换后形成的分析型数据表。由于预处理结果表字段较多，本文分三部分展示其示例数据，分别如图 4.5、图 4.6 和图 4.7 所示。

processed_job_details @shixiseng_d...							
开始事务 文本 筛选 排序 导入 导出 数据生成 创建图表							
group_name	item_text	job_title_detail	publish_time_detail	city_detail	degree_detail	deadline_detail	company_name_
产品	产品助理	产品助理	2026-01-06 22:29:06	南京	本科	2026-10-24	丁丁停车
产品	产品	产品经理	2026-03-18 17:30:13	上海	本科	2026-06-23	淘宝闪购
产品	产品经理	产品经理	2026-03-18 17:30:13	上海	本科	2026-06-23	淘宝闪购
产品	产品设计	产品设计实习生	2026-03-20 14:14:11	北京	本科	2026-05-30	知乎
产品	产品助理	产品助理实习生	2026-03-09 11:20:00	广州	本科	2026-09-05	ChatLabs
产品	产品	AI Agent产品经理	2026-03-18 17:30:10	上海	本科	2027-03-18	淘宝闪购
产品	产品经理	AI Agent产品经理	2026-03-18 17:30:10	上海	本科	2027-03-18	淘宝闪购
产品	产品设计	产品设计实习生	2026-03-14 00:00:36	上海	本科	2026-08-28	欧普照明
产品	产品助理	产品助理	2024-06-19 13:53:52	济南	大专	2026-03-31	济南柠欣
产品	产品设计	产品设计实习生	2026-03-12 14:53:46	上海	本科	2026-03-31	上海京萃
产品	产品经理	产品经理实习生	2025-11-06 13:58:07	成都	本科	2026-12-21	百词斩
产品	产品	产品部/产品实习生	2026-03-17 14:43:04	北京	本科	2026-12-30	滴滴
产品	产品	AI产品实习生	2025-09-30 16:09:14	北京	本科	2026-09-29	百融云创
产品	产品经理	产品经理实习生	2023-07-10 16:08:44	杭州	本科	2026-03-31	知米英语
产品	产品设计	产品设计实习生-TikTok设计	2026-03-17 15:14:26	上海	本科	2026-04-12	字节跳动
产品	产品助理	产品助理	2024-06-19 13:53:17	济南	本科	2026-03-31	神策科技
产品	产品	产品实习生	2023-10-16 10:46:36	武汉	本科	2026-03-31	人人网
产品	产品经理	产品经理实习生	2026-03-12 17:33:54	上海	本科	2026-11-08	得物App
产品	产品设计	产品设计实习生-TikTok设计	2026-03-17 15:13:58	上海	本科	2026-04-12	字节跳动
产品	产品助理	产品助理	2024-03-07 16:13:56	上海	本科	2026-03-31	海镜
产品	产品	AI产品实习生	2026-03-17 16:22:06	北京	本科	2027-01-01	百度
产品	产品经理	产品经理实习生	2026-03-08 20:00:26	上海	本科	2026-10-10	小红书
产品	产品设计	产品设计实习生-TikTok设计	2026-03-17 15:13:32	上海	本科	2026-04-12	字节跳动
产品	产品助理	产品助理	2023-05-02 15:24:21	北京	硕士	2026-03-31	道可道也
产品	产品	AI创作工具产品实习生	2026-01-22 10:22:23	上海	本科	2026-03-27	美团
产品	产品	AI创作工具产品实习生	2026-01-22 10:22:23	上海	本科	2026-03-27	美团
产品	产品经理	产品经理实习生	2026-02-28 14:14:46	深圳	本科	2026-03-25	OPPO
产品	产品助理	产品助理	2025-10-26 10:51:46	北京	不限	2026-03-31	12 Tech
产品	产品设计	产品设计实习生-TikTok设计	2026-03-17 15:13:28	北京	本科	2026-04-12	字节跳动

图 4.5 预处理结果表示例数据图（1）

processed_job_details @shixiseng_d...							
开始事务 文本 筛选 排序 导入 导出 数据生成 创建图表							
industry_detail	company_nature_detail	company_size_detail	company_location_detail	is_urgent_hire	job_tags	is_salary_negotiable	salary_min_p
互联网/游戏/软件	民营企业	50-150人	江苏		0 ["可转正实习", "接受大一"]		0
互联网/游戏/软件	民营企业	2000人以上	上海市/上海		1 ["周末双休"]		1
互联网/游戏/软件	民营企业	2000人以上	上海市/上海		1 ["周末双休"]		1
互联网/游戏/软件	民营企业	2000人以上	北京市		0 ["学习机会多", "成长空间大"]		0
互联网/游戏/软件	民营企业	50-150人	广东/广州		0 ["地铁周边", "周末双休", "一"]		0
互联网/游戏/软件	民营企业	2000人以上	上海市/上海		1 ["周末双休"]		1
互联网/游戏/软件	民营企业	2000人以上	上海市/上海		1 ["周末双休"]		1
快消/百货/批发/零售	股份制企业	2000人以上	上海市		0 ["地铁周边", "零基础实习", "一"]		0
互联网/游戏/软件	民营企业	50-150人	山东/济南		0 ["暑期实习", "零基础实习"]		0
快消/百货/批发/零售	民营企业	15-50人	上海市		0 ["不加班", "远程实习", "节E"]		0
互联网/游戏/软件	民营企业	500-2000人	四川		0 ["互联网公司", "福利好"]		0
互联网/游戏/软件	民营企业	2000人以上	全国		0 ["实习证明", "晚餐补贴", "京"]		0
金融/经济/投资/财会	股份制企业	500-2000人	北京市		0 ["AI产品", "1v1带教", "跨区"]		0
互联网/游戏/软件	民营企业	15-50人	杭州		0 ["产品好", "领导nice"]		0
互联网/游戏/软件	民营企业	2000人以上	全国		0 ["团队氛围好", "晋升空间大"]		0
互联网/游戏/软件	民营企业	15-50人	北京		0 ["可转正实习"]		0
互联网/游戏/软件	民营企业	2000人以上	湖北/武汉		0 ["定期团建"]		0
互联网/游戏/软件	民营企业	2000人以上	上海市/上海		0 ["餐补", "周末双休"]		0
互联网/游戏/软件	民营企业	2000人以上	全国		0 ["团队氛围好", "弹性工作", "餐补"]		0
互联网/游戏/软件	外资企业	15-50人	上海市/上海		0 ["接受大一大二", "留学生实"]		0
互联网/游戏/软件	外资企业	2000人以上	北京		0 ["一对一带教"]		0
互联网/游戏/软件	民营企业	2000人以上	上海市/上海		0 ["弹性工作", "免费三餐", "餐补"]		0
互联网/游戏/软件	民营企业	2000人以上	全国		0 ["团队氛围好", "晋升空间大"]		0
互联网/游戏/软件	民营企业	50-150人	北京		0 ["暑期实习"]		0
互联网/游戏/软件	民营企业	2000人以上	北京市/北京		0 ["餐补", "地铁周边", "周末双休"]		0
互联网/游戏/软件	民营企业	2000人以上	北京市/北京		0 ["餐补", "地铁周边", "周末双休"]		0
电子/通信/硬件	民营企业	2000人以上	深圳		0 ["地铁周边", "周末双休"]		0
互联网/游戏/软件	民营企业	15-50人	海外		0 ["初创", "腾讯背景创始人", "餐补"]		0
互联网/游戏/软件	民营企业	2000人以上	全国		0 ["团队氛围好", "弹性工作", "餐补"]		0

图 4.6 预处理结果表示例数据图（2）

is_urgent_hire	job_tags	is_salary_negotiable	salary_min_per_day	salary_max_per_day	internship_months	days_per_week
0	["可转正实习", "接受大一-大"]	0	150	300	3	4
1	["周末双休"]	1	(Null)	(Null)	3	5
1	["周末双休"]	1	(Null)	(Null)	3	5
0	["学习机会多", "成长空间大"]	0	120	150	3	5
0	["地铁周边", "周末双休", "一"]	0	120	140	3	4
1	["周末双休"]	1	(Null)	(Null)	3	5
1	["周末双休"]	1	(Null)	(Null)	3	5
0	["地铁周边", "零基础实习", ""]	0	150	180	6	5
0	["暑期实习", "零基础实习"]	0	100	100	3	5
0	["不加班", "远程实习", "节E"]	0	100	200	3	5
0	["互联网公司", "福利好"]	0	150	150	6	5
0	["实习证明", "晚餐补贴", "角"]	0	200	250	6	5
0	["AI产品", "1v1带教", "跨匠"]	0	150	200	6	5
0	["产品好", "领导nice"]	0	100	200	3	4
0	["团队氛围好", "晋升空间大"]	0	350	350	4	4
0	["可转正实习"]	0	150	200	3	5
0	["定期团建"]	0	100	150	3	5
0	["餐补", "周末双休"]	0	200	200	3	5
0	["团队氛围好", "弹性工作", ""]	0	350	350	4	4
0	["接受大一-大二", "留学生实"]	0	150	250	6	4
0	["一对一带教"]	0	250	300	3	5
0	["弹性工作", "免费三餐", "信"]	0	260	310	3	5
0	["团队氛围好", "晋升空间大"]	0	350	350	4	4
0	["暑期实习"]	0	100	200	3	5
0	["餐补", "地铁周边", "周末无"]	0	200	250	3	5
0	["餐补", "地铁周边", "周末无"]	0	200	250	3	5
0	["地铁周边", "周末双休"]	0	150	250	6	5
0	["初创", "腾讯背景创始人", ""]	0	80	100	4	4
0	["团队氛围好", "弹性工作", ""]	0	350	350	3	4

图 4.7 预处理结果表示例数据图（3）

由图 4.5 至图 4.7 可以看出，processed_job_details 表在原始详情表基础上删除了部分冗余字段，并新增了 is_urgent_hire、is_salary_negotiable、salary_min_per_day、salary_max_per_day、internship_months、days_per_week 等结构化变量，同时对岗位标签字段进行了 JSON 化处理。因此，该表在字段组织上更加紧凑，在数据类型上更加规范，也更适合作为后续 SQL 提取与问题分析的直接数据来源。

由此可见，本课题中的数据库并不仅仅承担保存抓取结果的作用，同时还承担了任务调度、状态记忆、原始数据沉淀和预处理结果支撑等多项功能。这种存储设计既保证了前端采集流程的可恢复性，也为后续的数据分析提供了更加稳定、规范的数据基础。

5 数据提取

5.1 问题一：不同岗位方向的数量分布如何

互联网 IT 类实习岗位包含多个具体岗位方向，不同岗位方向的岗位数量能够反映当前实习市场中不同技术方向的需求热度。因此，本节基于预处理结果表 `processed_job_details` 中的 `item_text` 字段进行分组统计，提取不同岗位方向对应的岗位数量，并按照岗位数量降序排列。提取数据命令如图 5.1 所示，执行后的结果如图 5.2 所示。

```
1 SELECT
2     item_text AS 岗位方向,
3     COUNT(*) AS 岗位数量
4 FROM processed_job_details
5 WHERE item_text IS NOT NULL
6     AND item_text <> ''
7 GROUP BY item_text
8 ORDER BY 岗位数量 DESC
9 LIMIT 20;
```

图 5.1 问题一数据提取命令图

岗位方向	岗位数量	岗位方向	岗位数量
产品运营	7517	海外运营	6144
数据运营	6935	用户运营	6109
运营助理	6770	网站运营	6008
游戏运营	6408	社群运营	5955
SEO/SEM运营	6390	运营	5741
品类运营	6356	产品助理	5567
内容运营	6293	产品项目管理	4907
活动运营	6202	产品经理	4375
运营专员	6182	产品	4300
电商运营	6165	新媒体运营	3169

图 5.2 问题一查询结果图

从查询结果可以看出，不同岗位方向之间的数量分布存在明显差异。其中，岗位数量最多的是“产品运营”，共有 7517 条记录；其次是“数据运营”，共有 6935 条记录；“运营助理”位居第三，共有 6770 条记录。除此之外，“游戏运营”为 6408 条，“SEO/SEM 运营”为 6390 条，“品类运营”为 6356 条，“内容运营”为 6293 条，“活动运营”为 6202 条，“运营专员”为 6182 条，“电商运营”为 6165 条。

从前 20 个岗位方向来看，数量较多的岗位主要集中在运营类和产品类方向，例如产品运营、数据运营、用户运营、网站运营、社群运营、产品助理、产品项目管

理和产品经理等。其中，前 15 个岗位方向的岗位数量均超过 5700 条，说明平台中互联网 IT 类实习岗位并不只集中在纯技术开发方向，运营类和产品类岗位同样占据较大比例。该结果表明，互联网 IT 实习岗位的需求结构具有一定的复合性，既包括技术相关岗位，也包括围绕互联网产品、用户增长和内容运营展开的业务型岗位。

5.2 问题二：互联网 IT 类实习岗位主要集中在哪些城市

城市分布能够反映互联网 IT 类实习岗位在空间上的集中程度。由于不同城市的产业基础、互联网企业数量和数字经济发展水平存在差异，实习岗位数量也会呈现出明显的区域差异。因此，本节基于 processed_job_details 表中的 city_detail 字段进行统计，提取不同城市对应的岗位数量，并按照岗位数量降序排列。提取数据命令如图 5.3 所示，执行后的结果如图 5.4 所示。

```
1 SELECT
2     city_detail AS 城市,
3     COUNT(*) AS 岗位数量
4 FROM processed_job_details
5 WHERE city_detail IS NOT NULL
6     AND city_detail <> ''
7 GROUP BY city_detail
8 ORDER BY 岗位数量 DESC
9 LIMIT 20;
```

图 5.3 问题二数据提取命令图

城市	岗位数量	城市	岗位数量
北京	57242	重庆	1502
上海	53003	长沙	1209
深圳	11133	无锡	1068
广州	9564	天津	1055
杭州	7792	西安	963
成都	6681	全国	930
苏州	2979	郑州	871
南京	2878	济南	650
武汉	2768	厦门	616
合肥	1722	佛山	538

图 5.4 问题二查询结果图

从查询结果可以看出，互联网 IT 类实习岗位在城市分布上具有明显的集中性。其中，北京的岗位数量最多，共有 57242 条记录；上海位居第二，共有 53003 条记录；深圳位居第三，共有 11133 条记录；广州位居第四，共有 9564 条记录；杭州位居第五，共有 7792 条记录；成都位居第六，共有 6681 条记录。

从数据差异来看，北京和上海的岗位数量明显高于其他城市，仅北京和上海两地合计就达到 110245 条记录，远高于深圳、广州、杭州、成都等城市。这说明互联

网 IT 类实习岗位高度集中在一线城市，尤其是北京和上海两地。深圳、广州、杭州和成都虽然岗位数量低于北京、上海，但仍明显高于多数其他城市，说明这些地区也具有较强的互联网产业基础和实习岗位供给能力。

此外，苏州、南京、武汉、合肥、重庆、长沙、无锡、天津、西安等城市也有一定数量的岗位发布，但整体数量与北京、上海相比存在较大差距。该结果说明，互联网 IT 实习岗位的空间分布呈现出“头部城市高度集中、其他城市梯度递减”的特征。

5.3 问题三：高薪岗位通常具有什么特征

薪资水平是衡量岗位吸引力的重要指标。为了分析高薪岗位的基本特征，本节基于 `processed_job_details` 表中的 `salary_min_per_day`、`salary_max_per_day` 和 `is_salary_negotiable` 字段进行数据提取。首先排除薪资面议以及薪资缺失的岗位记录，然后计算每条岗位的平均日薪，并利用 `NTILE(4)` 窗口函数将岗位按照平均日薪从高到低划分为四组。其中，平均日薪位于前 25% 的岗位被视为高薪岗位。提取数据命令如图 5.5 所示，执行后的结果如图 5.6 所示。

```
1 WITH salary_data AS (  
2     -- 构造薪资基础数据集  
3     -- 只保留薪资不是“面议”、且最低日薪和最高日薪均不为空的岗位记录  
4     SELECT  
5         item_text,                -- 岗位方向，如后端开发、数据分析、测试等  
6         city_detail,              -- 岗位所在城市  
7         internship_months,        -- 实习月数，由实习时长字段提取得到  
8         days_per_week,           -- 每周到岗天数，由每周工作天数字段提取得到  
9         salary_min_per_day,       -- 最低日薪  
10        salary_max_per_day,       -- 最高日薪  
11  
12        -- 计算平均日薪，用于衡量岗位薪资水平  
13        -- 计算方式为：（最低日薪 + 最高日薪） / 2  
14        ROUND((salary_min_per_day + salary_max_per_day) / 2, 2) AS salary_avg_per_day  
15    FROM processed_job_details  
16    WHERE is_salary_negotiable = 0 -- 排除薪资面议岗位  
17        AND salary_min_per_day IS NOT NULL -- 排除最低日薪缺失的记录  
18        AND salary_max_per_day IS NOT NULL -- 排除最高日薪缺失的记录  
19 ),  
20  
21 ranked_salary AS (  
22     -- 按照平均日薪从高到低进行分组排序  
23     -- NTILE(4) 表示将全部有效薪资岗位划分为 4 组  
24     -- salary_level = 1 表示平均日薪最高的前 25% 岗位，即高薪岗位  
25     SELECT  
26         *,  
27         NTILE(4) OVER (ORDER BY salary_avg_per_day DESC) AS salary_level  
28     FROM salary_data  
29 )  
30  
31 -- 提取高薪岗位明细  
32 -- 只保留 salary_level = 1 的记录，即平均日薪处于前 25% 的岗位  
33 SELECT  
34     item_text AS 岗位方向,  
35     city_detail AS 城市,  
36     internship_months AS 实习月数,  
37     days_per_week AS 每周到岗天数,  
38     salary_min_per_day AS 最低日薪,  
39     salary_max_per_day AS 最高日薪,  
40     salary_avg_per_day AS 平均日薪  
41 FROM ranked_salary  
42 WHERE salary_level = 1  
43 ORDER BY salary_avg_per_day DESC -- 按平均日薪从高到低排列  
44 LIMIT 10;                       -- 只展示前 10 条高薪岗位记录
```

图 5.5 问题三数据提取命令图

岗位方向	城市	实习月数	每周到岗天数	最低日薪	最高日薪	平均日薪
后端开发	上海	3	5	1500	2000	1750.00
人工智能	北京	3	5	1500	2000	1750.00
数据运营	北京	3	5	1500	2000	1750.00
人工智能	上海	3	5	1500	2000	1750.00
搜索算法	上海	3	5	1500	2000	1750.00
数据	北京	3	5	1500	2000	1750.00
算法研究员	上海	3	5	1500	2000	1750.00
图像算法	上海	3	5	1500	2000	1750.00
大数据开发工程师	北京	3	5	1500	2000	1750.00
规控算法	上海	3	5	1500	2000	1750.00

图 5.6 问题三查询结果图

从查询结果可以看出，高薪岗位的薪资水平较为集中。结果中展示的前 10 条高薪岗位，最低日薪均为 1500 元/天，最高日薪均为 2000 元/天，平均日薪均达到 1750 元/天，说明这些岗位处于样本中较高的薪资区间。

从岗位方向来看，前 10 条高薪岗位主要集中在后端开发、人工智能、数据运营、搜索算法、数据、算法研究员、图像算法、大数据开发工程师和规控算法等方向。其中，人工智能相关岗位在结果中出现 2 次，算法类岗位包括搜索算法、算法研究员、图像算法和规控算法，说明人工智能、算法、数据和后端开发等方向更容易出现在高薪岗位中。

从城市分布来看，前 10 条高薪岗位主要集中在北京和上海。其中，上海出现的岗位包括后端开发、人工智能、搜索算法、算法研究员、图像算法和规控算法等；北京出现的岗位包括人工智能、数据运营、数据和大数据开发工程师等。这说明高薪岗位不仅与岗位技术方向有关，也与城市互联网产业集聚程度密切相关。

从实习要求来看，前 10 条高薪岗位的实习月数均为 3 个月，每周到岗天数均为 5 天。这说明高薪岗位通常要求较稳定的实习周期和较高的到岗频率，企业在提供较高薪资的同时，也对实习生的时间投入和岗位参与度提出了较高要求。总体来看，高薪岗位主要集中在北京、上海等城市，并以人工智能、算法、数据和后端开发等技术方向为主，具有薪资高、到岗要求明确、实习周期相对稳定的特征。

5.4 问题四：企业对技术技能的要求主要体现在哪些关键词上

技术技能关键词能够反映企业在互联网 IT 类实习岗位中对求职者能力的关注重点。在预处理阶段，原始岗位标签字段已经被整理为 `job_tags` 字段，因此本节基于

该字段进行关键词匹配统计。通过 SQL 模糊匹配方式，分别统计 Python、SQL、Java、前端、后端、数据分析、机器学习、深度学习、人工智能和 AI Agent 等关键词在岗位标签中的出现岗位数量，并按照出现数量降序排列。提取数据命令如图 5.7 所示，执行后的结果如图 5.8 所示。

```
1 SELECT 技能关键词, 出现岗位数量
2 FROM (
3     SELECT 'Python' AS 技能关键词, COUNT(*) AS 出现岗位数量
4     FROM processed_job_details
5     WHERE job_tags LIKE '%Python%'
6
7     UNION ALL
8
9     SELECT 'SQL' AS 技能关键词, COUNT(*) AS 出现岗位数量
10    FROM processed_job_details
11    WHERE job_tags LIKE '%SQL%'
12
13    UNION ALL
14
15    SELECT 'Java' AS 技能关键词, COUNT(*) AS 出现岗位数量
16    FROM processed_job_details
17    WHERE job_tags LIKE '%Java%'
18
19    UNION ALL
20
21    SELECT '前端' AS 技能关键词, COUNT(*) AS 出现岗位数量
22    FROM processed_job_details
23    WHERE job_tags LIKE '%前端%'
24        OR job_tags LIKE '%HTML%'
25        OR job_tags LIKE '%CSS%'
26        OR job_tags LIKE '%JavaScript%'
27        OR job_tags LIKE '%Vue%'
28        OR job_tags LIKE '%React%'
29
30    UNION ALL
31
32    SELECT '后端' AS 技能关键词, COUNT(*) AS 出现岗位数量
33    FROM processed_job_details
34    WHERE job_tags LIKE '%后端%'
35
36    UNION ALL
37
38    SELECT '数据分析' AS 技能关键词, COUNT(*) AS 出现岗位数量
39    FROM processed_job_details
40    WHERE job_tags LIKE '%数据分析%'
41
42    UNION ALL
43
44    SELECT '深度学习' AS 技能关键词, COUNT(*) AS 出现岗位数量
45    FROM processed_job_details
46    WHERE job_tags LIKE '%深度学习%'
47
48    UNION ALL
49
50    SELECT '人工智能' AS 技能关键词, COUNT(*) AS 出现岗位数量
51    FROM processed_job_details
52    WHERE job_tags LIKE '%人工智能%'
53        OR job_tags LIKE '%AI%'
54
55    UNION ALL
56
57    SELECT 'AI Agent' AS 技能关键词, COUNT(*) AS 出现岗位数量
58    FROM processed_job_details
59    WHERE job_tags LIKE '%AI Agent%'
60
61    UNION ALL
```

图 5.7 问题四数据提取命令图

技能关键词	出现岗位数量
人工智能	1594
数据分析	644
Python	198
SQL	148
机器学习	84
AI Agent	83
深度学习	82
前端	80
Java	63
后端	31

图 5.8 问题四查询结果图

从查询结果可以看出，不同技术关键词的出现岗位数量存在明显差异。其中，出现数量最多的关键词是“人工智能”，共有 1594 条岗位记录；其次是“数据分析”，共有 644 条记录；“Python”位居第三，共有 198 条记录；“SQL”共有 148 条记录；“机器学习”共有 84 条记录；“AI Agent”共有 83 条记录；“深度学习”共有 82 条记录；“前端”共有 80 条记录；“Java”共有 63 条记录；“后端”共有 31 条记录。

从结果来看，人工智能类关键词的出现数量明显高于其他技能关键词，说明在当前互联网 IT 类实习岗位标签中，人工智能相关能力具有较高关注度。同时，数据分析关键词出现 644 次，说明数据处理、数据理解 and 数据分析能力也是企业较为重视的技能方向。Python 和 SQL 分别出现 198 次和 148 次，表明编程语言和数据库查询能力仍是互联网 IT 实习岗位中的基础技能要求。

此外，机器学习、深度学习和 AI Agent 的出现数量分别为 84 条、82 条和 83 条，说明人工智能细分方向已经在部分实习岗位中有所体现。相比之下，前端、Java 和后端关键词的出现数量相对较少，分别为 80 条、63 条和 31 条。这一结果说明，在当前岗位标签统计范围内，企业对人工智能、数据分析、Python 和 SQL 等方向的显性标注更为突出，而传统前后端开发类关键词在本次关键词匹配结果中的数量相对较低。总体来看，技能关键词统计结果反映出互联网 IT 实习岗位对人工智能、数据分析和基础编程能力具有较强需求。

5.5 问题五：岗位平均日薪分布情况

薪资水平是衡量岗位吸引力的重要指标。为了进一步分析互联网 IT 类实习岗位的薪资分布，本节基于预处理结果表 `processed_job_details` 中的 `salary_min_per_day` 和 `salary_max_per_day` 字段计算每条岗位的平均日薪，并结合岗位方向与城市进行统计分析。为保证统计可靠性，首先排除薪资面议或缺失的岗位记录，即 `is_salary_negotiable = 0` 且 `salary_min_per_day` 与 `salary_max_per_day` 均不为空。提取数据命令如图 5.9 所示，执行后的结果如图 5.10 所示。

```

1 WITH city_job_salary AS (
2     -- 统计每个城市、每个岗位方向的平均日薪、岗位数量
3     SELECT
4         city_detail AS 城市,
5         item_text AS 岗位方向,
6         COUNT(*) AS 岗位数量,
7         ROUND(AVG((salary_min_per_day + salary_max_per_day) / 2), 2) AS 平均日薪
8     FROM processed_job_details
9     WHERE is_salary_negotiable = 0 -- 排除薪资面议
10        AND salary_min_per_day IS NOT NULL -- 排除无最低日薪
11        AND salary_max_per_day IS NOT NULL -- 排除无最高日薪
12        AND city_detail IS NOT NULL
13        AND item_text IS NOT NULL
14    GROUP BY city_detail, item_text
15 )
16
17 -- 岗位数量≥3、薪资从高到低的城市-岗位组合
18 SELECT
19     城市,
20     岗位方向,
21     岗位数量,
22     平均日薪
23 FROM city_job_salary
24 WHERE 岗位数量 >= 3
25 ORDER BY 平均日薪 DESC
26 LIMIT 20;

```

图 5.9 问题五数据提取命令图

城市	岗位方向	岗位数量	平均日薪
全国	图像算法	4	481.25
全国	规控算法	4	481.25
全国	搜索算法	4	481.25
全国	自然语言处理算法	4	481.25
全国	SLAM算法	4	481.25
全国	语音算法	4	481.25
全国	推荐算法	4	481.25
全国	人工智能	11	472.73
全国	通信测试	3	441.67
全国	软件测试	3	433.33
全国	数据挖掘	8	415.62
全国	算法研究员	5	412.00
全国	风控算法	5	410.00
全国	算法工程师	17	408.24
全国	数据	9	402.78
全国	Python	10	397.50
北京	iOS	10	380.00
北京	机器学习	12	379.17
全国	大模型算法	6	379.17
深圳	Android	5	370.00

图 5.10 问题五查询结果图

从查询结果可以看出，高薪岗位主要集中在算法类和人工智能方向，例如图像算法、规控算法、搜索算法、自然语言处理算法、SLAM 算法、语音算法、推荐算法和人工智能岗位，其平均日薪均超过 470 元/天。其中，算法类岗位虽然岗位数量较少（岗位数量为 4），但薪资水平最高，说明企业在关键技术岗位上提供了更高的薪酬。人工智能方向岗位数量较多（岗位数量为 11），平均日薪略低于算法类岗位，但仍位于样本前列。

从城市分布来看，北京和上海仍然占据高薪岗位的主要集中地。例如，北京的 iOS 与机器学习岗位以及深圳的 Android 岗位均进入前 20 名，说明城市产业集聚对岗位薪资水平具有明显影响。同时，也存在部分高薪岗位呈全国分布特征，如算法类岗位和人工智能岗位。

从岗位数量与薪资关系来看，岗位数量较少的关键技术岗位往往薪资更高，而数量较多的常规技术岗位薪资略低，反映出企业在有限资源下对关键岗位的高薪倾斜。

总体来看，互联网 IT 类实习岗位的平均日薪分布集中在高技能技术岗位，薪资水平与岗位方向和城市位置密切相关。该结果可为学生选择实习岗位提供参考，并可结合第 6 章可视化分析进一步呈现薪资差异的空间与方向分布特征。

6 数据可视化与分析

6.1 岗位方向数量分布可视化分析

为分析互联网 IT 类实习岗位在不同岗位方向上的数量差异，本节基于预处理结果表 `processed_job_details` 中的 `item_text` 字段进行统计。代码首先对 `item_text` 字段进行空值填充、字符串转换和首尾空格清理，然后按照岗位方向进行分组统计，并选取岗位数量排名前 20 的岗位方向作为可视化对象。为了便于比较不同岗位方向之间的数量差异，本节采用横向条形图进行展示。对应代码如图 6.1 所示，岗位方向数量分布结果如图 6.2 所示。

```
[5]: 1 df["item_text"] = df["item_text"].fillna("").astype(str).str.strip()
2
3 df_job_direction = (
4     df[df["item_text"] != ""]
5     .groupby("item_text", as_index=False)
6     .size()
7     .rename(columns={"item_text": "岗位方向", "size": "岗位数量"})
8     .sort_values("岗位数量", ascending=False)
9     .head(20)
10 )
11
12 df_plot = df_job_direction.sort_values("岗位数量", ascending=True)
13
14 plt.figure(figsize=(11, 8))
15
16 ax = sns.barplot(data=df_plot, x="岗位数量", y="岗位方向", color="#4C72B0")
17
18 for container in ax.containers:
19     ax.bar_label(container, fmt="{:.0f}", padding=3, fontsize=9)
20
21 # plt.title("不同岗位方向岗位数量分布", fontsize=17, fontweight="bold", pad=15)
22 plt.xlabel("岗位数量", fontsize=12)
23 plt.ylabel("岗位方向", fontsize=12)
24 plt.grid(axis="x", linestyle="--", alpha=0.35)
25 sns.despine(left=True, bottom=True)
26 plt.tight_layout()
27 plt.show()
```

图 6.1 岗位方向数量分布可视化代码图

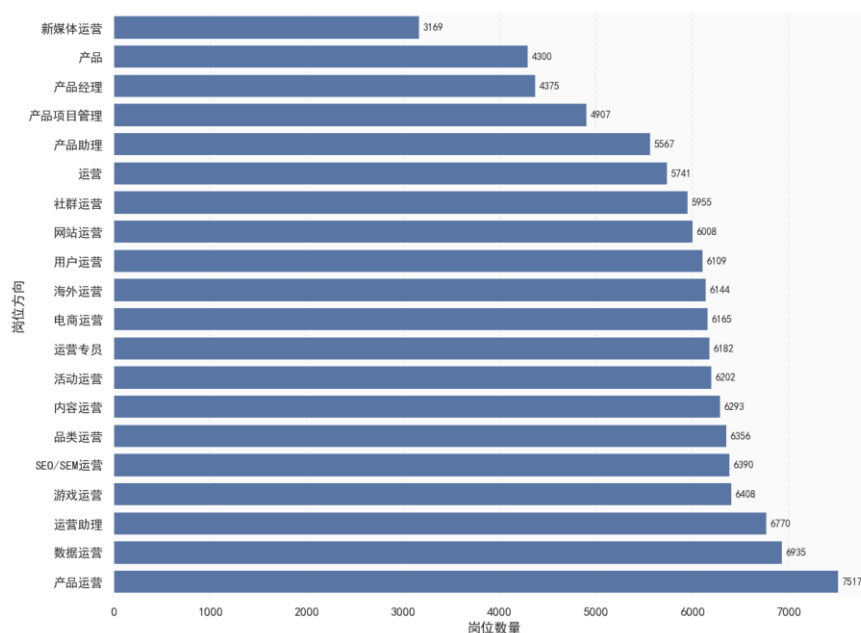


图 6.2 不同岗位方向岗位数量分布图

从图 6.2 可以看出，互联网 IT 类实习岗位在不同方向之间存在明显差异。其中，“产品运营”的岗位数量最多，共有 7517 条记录；其次是“数据运营”，共有 6935 条记录；“运营助理”位居第三，共有 6770 条记录。排名靠前的岗位还包括“游戏运营”6408 条、“SEO/SEM 运营”6390 条、“品类运营”6356 条、“内容运营”6293 条、“活动运营”6202 条、“运营专员”6182 条和“电商运营”6165 条。

从整体结构来看，前 20 个岗位方向中，运营类岗位占据较大比例，说明实习僧平台互联网 IT 类实习岗位并不只集中在技术开发方向，而是更多体现出产品运营、数据运营、内容运营、用户运营、社群运营等互联网业务岗位的需求特征。同时，“产品助理”“产品项目管理”“产品经理”等产品类岗位也具有一定数量，说明产品相关岗位也是互联网 IT 类实习岗位中的重要组成部分。

由此可见，互联网 IT 类实习岗位的岗位方向呈现出明显的复合型结构，既包括技术相关岗位，也包括产品、运营和业务支持类岗位。该结果说明，在互联网行业实习岗位需求中，运营能力、产品理解能力和数据意识同样具有较高现实需求。

6.2 城市岗位数量分布可视化分析

为分析互联网 IT 类实习岗位在不同城市之间的分布情况，本节基于 `city_detail` 字段统计各城市对应的岗位数量。代码首先对城市字段进行空值处理和字符串清洗，并剔除“全国”等不适合用于城市维度分析的记录，然后按照城市进行分组统计，选取岗位数量排名前 20 的城市作为分析对象。为了避免继续使用普通柱状图造成图表形式重复，本节采用气泡图展示城市岗位数量差异。对应代码如图 6.3 所示，城市岗位数量分布结果如图 6.4 所示。

```
[7]: 1 df["city_detail"] = df["city_detail"].fillna("").astype(str).str.strip()
2
3 df_city = (
4     df[(df["city_detail"] != "") & (df["city_detail"] != "全国")].groupby("city_detail", as_index=False).size()
5     .rename(columns={"city_detail": "城市", "size": "岗位数量"}).sort_values("岗位数量", ascending=False).head(20)
6 )
7
8 df_plot = df_city.sort_values("岗位数量", ascending=True).copy()
9
10 plt.figure(figsize=(12, 8))
11
12 ax = sns.scatterplot(
13     data=df_plot, x="岗位数量", y="城市", size="岗位数量", sizes=(200, 3500),
14     alpha=0.65, color="#55A868", edgecolor="#2F5D50", linewidth=1, legend=False
15 )
16
17 for _, row in df_plot.iterrows():
18     ax.text(row["岗位数量"] + 500, row["城市"], f"{int(row['岗位数量'])}", va="center", fontsize=9)
19
20 # plt.title("互联网 IT 类实习岗位城市分布气泡图", fontsize=17, fontweight="bold", pad=15)
21 plt.xlabel("岗位数量", fontsize=12)
22 plt.ylabel("城市", fontsize=12)
23 plt.grid(axis="x", linestyle="--", alpha=0.35)
24 sns.despine(left=True, bottom=True)
25 plt.tight_layout()
26 plt.show()
```

图 6.3 城市岗位数量分布可视化代码图

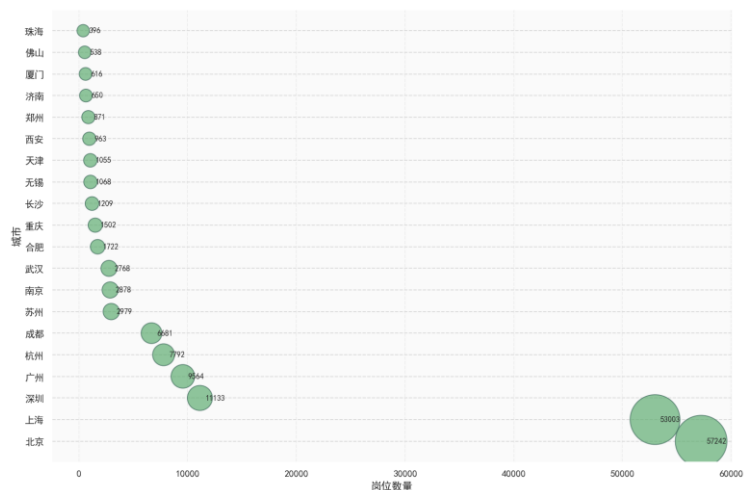


图 6.4 互联网 IT 类实习岗位城市分布气泡图

从图 6.4 可以看出，互联网 IT 类实习岗位具有非常明显的城市集中现象。其中，北京岗位数量最多，共有 57242 条记录；上海位居第二，共有 53003 条记录。北京和上海两个城市的气泡明显大于其他城市，说明这两个城市是互联网 IT 类实习岗位最主要的集中地区。

除北京和上海外，深圳、广州、杭州和成都也具有较高岗位数量。其中，深圳为 11133 条，广州为 9564 条，杭州为 7792 条，成都为 6681 条。这些城市虽然与北京、上海存在明显差距，但仍高于其他多数城市，说明其互联网产业基础和企业实习岗位供给能力相对较强。

从图中还可以看到，苏州、南京、武汉、合肥、重庆、长沙、无锡、天津、西安、郑州、济南、厦门、佛山、珠海等城市也存在一定数量的岗位发布，但整体规模明显低于头部城市。这说明互联网 IT 类实习岗位在空间分布上呈现出“头部城市高度集中、次级城市梯度分布”的特征。对于求职者而言，北京、上海、深圳、广州、杭州和成都仍是互联网 IT 实习机会较为集中的城市。

6.3 技能关键词需求可视化分析

为分析互联网 IT 类实习岗位中企业对技术能力的关注重点，本节基于 `job_tags` 字段对常见技能关键词进行匹配统计。代码中选取了“人工智能”“数据分析”“Python”“SQL”“机器学习”“AI Agent”“深度学习”“前端”“Java”“后端”等关键词，并通过字符串匹配统计每类关键词在岗位标签中的出现次数。为了展示不同技能关键词在整体关键词统计中的占比结构，本节使用环形图进行可视化。对应代码如图 6.5 所示，技能关键词占比结果如图 6.6 所示。

```
[9]: 1 df["job_tags"] = df["job_tags"].fillna("").astype(str).strip()
2
3 skill_patterns = {
4     "人工智能": ["人工智能", "AI"],
5     "数据分析": ["数据分析"],
6     "Python": ["Python", "python"],
7     "SQL": ["SQL", "sql"],
8     "机器学习": ["机器学习"],
9     "AI Agent": ["AI Agent", "Agent", "agent"],
10    "深度学习": ["深度学习"],
11    "前端": ["前端", "HTML", "CSS", "JavaScript", "Vue", "React"],
12    "Java": ["Java", "java"],
13    "后端": ["后端"]
14 }
15
16 skill_rows = []
17
18 for skill_name, patterns in skill_patterns.items():
19     pattern = "|".join([re.escape(p) for p in patterns])
20     count = df["job_tags"].str.contains(pattern, case=False, regex=True, na=False).sum()
21     skill_rows.append(("技能关键词": skill_name, "出现岗位数量": int(count)))
22
23 df_skill = (pd.DataFrame(skill_rows).sort_values("出现岗位数量", ascending=False).reset_index(drop=True))
24
25 plt.figure(figsize=(9, 9))
26
27 colors = sns.color_palette("Set3", n_colors=len(df_skill))
28
29 wedges, texts, autotexts = plt.pie(
30     df_skill["出现岗位数量"], labels=df_skill["技能关键词"], autopct="%1.1f%%", startangle=90,
31     pctdistance=0.82, textprops={"fontsize": 10}, colors=colors
32 )
33
34 centre_circle = plt.Circle((0, 0), 0.58, fc="white")
35 plt.gca().add_artist(centre_circle)
36
37 # plt.title("技能关键词占比环状图", fontsize=17, fontweight="bold", pad=20)
38 plt.tight_layout()
39 plt.show()
```

图 6.5 技能关键词占比可视化代码图

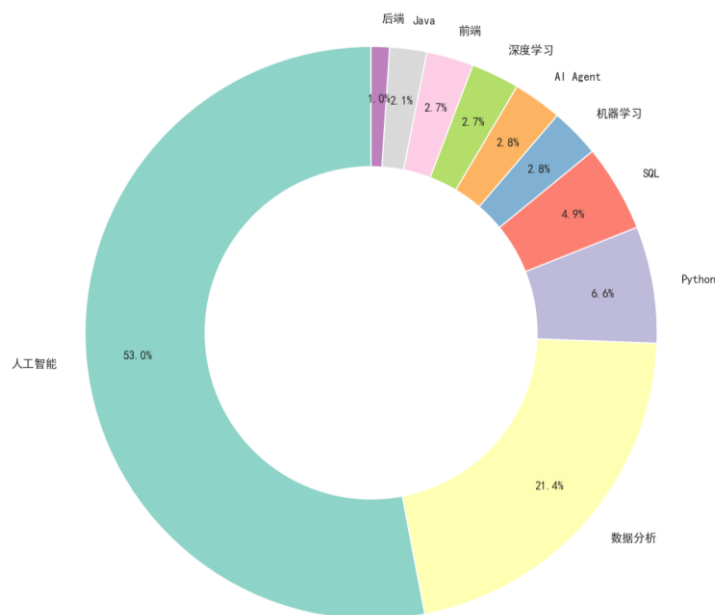


图 6.6 技能关键词占比环状图

从图 6.6 可以看出，“人工智能”关键词占比最高，达到 53.0%，在所有统计关键词中占据绝对优势；“数据分析”位居第二，占比为 21.4%。这说明在当前互联网 IT 类实习岗位标签中，人工智能和数据分析相关能力具有较强的显性需求。

从基础编程和数据处理技能来看，“Python”占比为 6.6%，“SQL”占比为 4.9%。这表明 Python 编程能力和 SQL 数据查询能力仍是互联网 IT 类实习岗位中较为常见的基础能力要求。与此同时，“机器学习”“AI Agent”“深度学习”等人工智能细分关键词也具有一定占比，其中“机器学习”为 2.8%，“AI Agent”为 2.8%，“深度学习”为

2.7%。这说明企业在部分实习岗位中已经开始体现对人工智能细分方向的需求。

相比之下，“前端”“Java”“后端”等传统开发类关键词占比较低，其中“前端”为 2.7%， “Java”为 2.1%， “后端”为 1.0%。这并不代表开发类岗位不重要，而是说明在本次基于 job_tags 的关键词统计中，人工智能、数据分析和数据处理相关标签更加突出。整体来看，互联网 IT 类实习岗位的技能需求呈现出智能化、数据化和基础编程能力并重的特点。

6.4 急招岗位数量与占比可视化分析

为分析不同岗位方向的短期用人紧迫程度，本节基于 is_urgent_hire 字段统计急招岗位情况。代码首先将 is_urgent_hire 字段转换为数值型变量，然后筛选岗位数量排名前 15 的岗位方向，并分别计算每个岗位方向的岗位总数、急招岗位数量和急招占比。为了同时展示“急招岗位数量”和“急招岗位占比”两个指标，本节采用双轴图进行展示，其中柱状图表示急招岗位数量，折线图表示急招占比。对应代码如图 6.7 所示，急招岗位数量与占比结果如图 6.8 所示。

```
[11]: 1 # 1. 急招字段转为数值型
2 df["is_urgent_hire"] = (pd.to_numeric(df["is_urgent_hire"], errors="coerce").fillna(0).astype(int))
3
4 # 2. 岗位方向字段清洗
5 df["item_text"] = (df["item_text"].fillna("").astype(str).strip())
6
7 # 3. 保留岗位方向不为空的数据
8 df_urgent_base = df[df["item_text"] != ""].copy()
9
10 # 4. 提取岗位数量排名前 15 的岗位方向
11 top_directions = (df_urgent_base.groupby("item_text").size().sort_values(ascending=False).head(15).index)
12
13 df_urgent_top = df_urgent_base[df_urgent_base["item_text"].isin(top_directions)].copy()
14
15 # 5. 统计每个岗位方向的岗位总数和急招岗位数量
16 df_urgent_dual = (
17     df_urgent_top.groupby("item_text").agg(total_jobs=("item_text", "size"), urgent_jobs=("is_urgent_hire", "sum"))
18     .reset_index().rename(columns={"item_text": "job_direction"})
19 )
20
21 # 6. 计算急招占比
22 df_urgent_dual["urgent_rate"] = (df_urgent_dual["urgent_jobs"] / df_urgent_dual["total_jobs"] * 100).round(2)
23
24 # 7. 按急招岗位数量降序排列
25 df_urgent_dual = (df_urgent_dual.sort_values("urgent_jobs", ascending=False).reset_index(drop=True))
26
27 # 8. 控制双轴图
28 x = range(len(df_urgent_dual))
29
30 fig, ax1 = plt.subplots(figsize=(14, 7))
31
32 bars = ax1.bar(x, df_urgent_dual["urgent_jobs"], width=0.6, color="#4C72B0", alpha=0.82, label="急招岗位数量")
33
34 ax1.set_xlabel("岗位方向", fontsize=12)
35 ax1.set_ylabel("急招岗位数量", fontsize=12)
36 ax1.set_xticks(x)
37 ax1.set_xticklabels(df_urgent_dual["job_direction"], rotations=45, ha="right")
38 ax1.grid(axis="y", linestyle="--", alpha=0.35)
39
40 for rect in bars:
41     height = rect.get_height()
42     ax1.text(rect.get_x() + rect.get_width() / 2, height, f"{int(height)}", ha="center", va="bottom", fontsize=9)
43
44 ax2 = ax1.twinx()
45
46 ax2.plot(x, df_urgent_dual["urgent_rate"], color="#C44E52", marker="o", linewidth=2.2, label="急招占比")
47
48 ax2.set_ylabel("急招占比 (%)", fontsize=12)
49
50 for i, value in enumerate(df_urgent_dual["urgent_rate"]):
51     ax2.text(i, value, f"{value:.2f}%", ha="center", va="bottom", fontsize=9, color="#8B1E3F")
52
53 handles1, labels1 = ax1.get_legend_handles_labels()
54 handles2, labels2 = ax2.get_legend_handles_labels()
55
56 ax1.legend(handles1 + handles2, labels1 + labels2, loc="upper right")
57
58 sns.despine(left=False, bottom=True)
59 plt.tight_layout()
60 plt.show()
```

图 6.7 各岗位方向急招岗位数量与占比可视化代码图

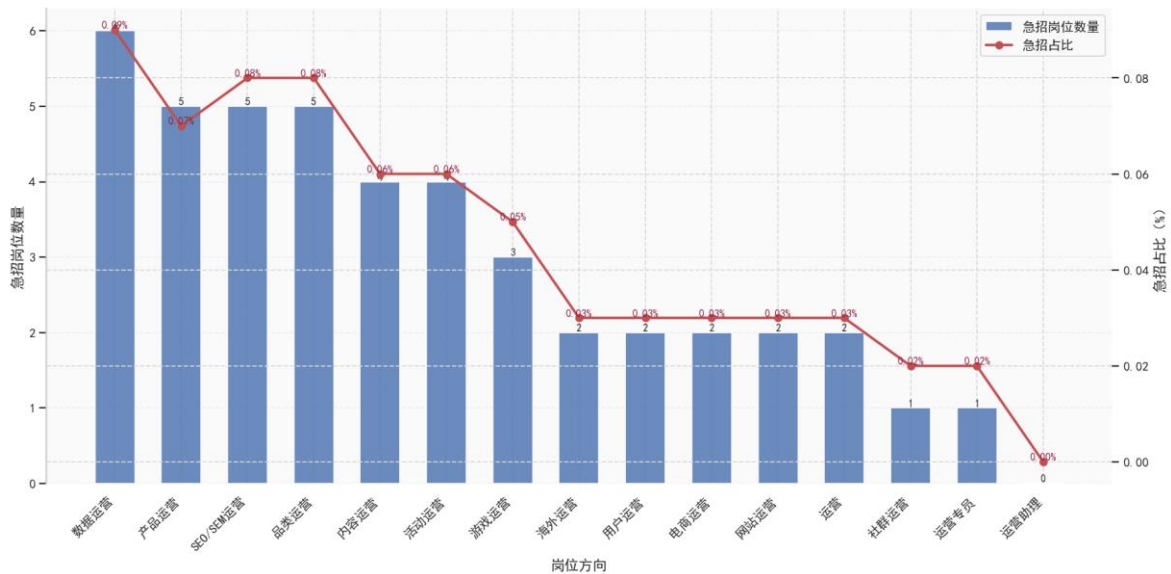


图 6.8 各岗位方向急招岗位数量与占比双轴图

从图 6.8 可以看出，不同岗位方向的急招岗位数量整体较少，但仍存在一定差异。其中，“数据运营”的急招岗位数量最多，共有 6 条，急招占比为 0.09%；“产品运营”的急招岗位数量为 5 条，急招占比为 0.07%；“SEO/SEM 运营”和“品类运营”的急招岗位数量也均为 5 条，急招占比分别约为 0.08%。

从中间梯队来看，“内容运营”和“活动运营”的急招岗位数量均为 4 条，急招占比约为 0.06%；“游戏运营”的急招岗位数量为 3 条，急招占比约为 0.05%。其余岗位方向，如“运营”“电商运营”“海外运营”“用户运营”“网站运营”等急招岗位数量多为 2 条，急招占比约为 0.03%。

从结果整体来看，急招岗位主要集中在数据运营、产品运营、SEO/SEM 运营、品类运营、内容运营、活动运营等运营相关方向。这说明运营类岗位不仅在整体岗位数量上占比较高，也更容易出现短期招聘需求。但需要注意的是，各岗位方向的急招占比整体都较低，最高也仅为 0.09%，说明“急招”岗位在全部样本中属于少量特殊情况，更多体现的是部分岗位方向的短期用人需求，而不是整体招聘市场的普遍状态。

6.5 岗位标签词云可视化分析

为进一步分析岗位标签中的高频内容，本节基于 `job_tags` 字段进行标签解析和词频统计。岗位标签属于半结构化文本信息，既包含技能要求，也包含实习福利、工作制度和成长支持等内容。对于这类文本字段，可以通过词频统计和文本可视化

方式呈现高频关键词特征^[16]。由于 job_tags 字段可能存在 JSON 字符串、Python 列表字符串和普通分隔字符串等不同形式，因此代码中分别尝试使用 json.loads、ast.literal_eval 和正则分割方式进行解析，并将解析出的全部标签汇总后使用 Counter 统计词频。最后，基于词频结果生成岗位标签词云图。对应代码如图 6.9 所示，岗位标签词云结果如图 6.10 所示。

```
[13]: 1 # 用于存放解析后的全部岗位标签
2 all_tags = []
3
4 # 遍历 job_tags 字段中的每一条记录
5 for value in df["job_tags"]:
6
7     # 如果当前值为空值，直接跳过
8     if pd.isna(value):
9         continue
10
11     # 如果当前值本身已经是列表，则直接逐个清洗后加入 all_tags
12     if isinstance(value, list):
13         all_tags.extend([str(x).strip() for x in value if str(x).strip()])
14         continue
15
16     # 将当前值转换为字符串，并去除首尾空格
17     text = str(value).strip()
18
19     # 如果转换为空字符串，直接跳过
20     if not text:
21         continue
22
23     # 标记当前文本是否已经被成功解析
24     parsed_flag = False
25
26     try:
27         parsed = json.loads(text)
28
29         # 如果解析结果是列表，则将列表中的标签清洗后加入 all_tags
30         if isinstance(parsed, list):
31             all_tags.extend([str(x).strip() for x in parsed if str(x).strip()])
32             parsed_flag = True
33     except Exception:
34         # JSON 解析失败时不报错，继续尝试下一种解析方式
35         pass
36
37     # 如果已经成功解析，则进入下一条记录
38     if parsed_flag:
39         continue
40
41     # 如果已经成功解析，则进入下一条记录
42     if parsed_flag:
43         continue
44
45     # 将拆分后的非空标签加入 all_tags
46     all_tags.extend([p.strip() for p in parts if p.strip()])
47
48 # 统计所有岗位标签的出现频次
49 tag_counter = Counter(all_tags)
50
51 font_candidates = ["C:\\Windows\\Fonts\\simhei.ttf", "C:\\Windows\\Fonts\\msyh.ttf", "C:\\Windows\\Fonts\\simsun.ttf"]
52
53 # 保存最终可用的字体路径
54 font_path = None
55
56 # 从候选字体中查找第一个存在的字体文件
57 for fp in font_candidates:
58     if os.path.exists(fp):
59         font_path = fp
60         break
61
62 # 如果未找到中文字体，则主动报错，提示手动设置字体路径
63 if font_path is None:
64     raise FileNotFoundError("未找到中文字体文件，请手动设置 font_path")
65
66 # 生成词云对象
67 wordcloud = WordCloud(
68     font_path=font_path, width=1400, height=900, background_color="white", max_words=180,
69     collocations=False, contour_width=1, contour_color="#DDDDDD"
70 ).generate_from_frequencies(tag_counter)
71
72
73 plt.figure(figsize=(13, 8)) # 绘制词云图
74 plt.imshow(wordcloud, interpolation="bilinear") # 使用双线性插值显示词云，使图像边缘更平滑
75 plt.axis("off") # 关闭坐标轴，使图像更适合报告展示
76 plt.tight_layout()
77 plt.show()
```

图 6.9 岗位标签词云可视化代码图



从图 6.10 可以看出，岗位标签中的高频内容主要集中在实习福利、工作制度、成长支持和岗位环境等方面。其中，“周末双休”“地铁周边”“可转正实习”“餐补”“实习津贴”“一对一导师”等词语在词云中占据较大面积，说明这些标签在互联网 IT 类实习岗位中出现频率较高。

此外，“一对一导师”“团队氛围好”“零基础实习”“接受大一大二”“弹性工作”“不加班”等标签也较为明显。这说明互联网 IT 类实习岗位在招聘表达上不仅关注岗位职责和技能要求，也较重视实习体验、成长机会和工作环境。对于学生群体而言，这些标签能够直接影响岗位吸引力；对于企业而言，这些标签也是提升岗位竞争力的重要展示内容。整体来看，岗位标签词云反映出互联网 IT 类实习岗位具有明显的福利导向、成长导向和体验导向。

7 总结

本次课程设计围绕“基于实习僧平台的互联网 IT 类实习岗位的数据采集与分析”这一主题，完成了从任务构建、岗位抓取、数据预处理、数据库存储到数据提取分析的完整流程。整体来看，本课题不是简单地保存网页信息，而是构建了一套较为完整的数据采集与处理流程。

在数据爬取方面，系统首先从实习僧首页“互联网 IT”菜单中提取岗位分类，并生成标准化搜索任务；随后通过主抓取程序批量访问岗位列表页和详情页，提取岗位标题、城市、薪资、实习要求、岗位标签等字段。程序采用多进程协同、双标签页、URL 规范化、联合键去重和进度记录等机制，提高了抓取效率和稳定性。

在数据预处理方面，课题对原始岗位详情数据进行了字段清理和特征构造，删除了与分析关系较弱的冗余字段，并生成了急招标志、结构化岗位标签、最低日薪、最高日薪、薪资面议标志、实习月数和每周到岗天数等分析变量，使原始网页数据转化为更适合统计分析的结构化数据。

在数据存储方面，系统使用 MySQL 对任务数据、原始岗位详情、预处理结果和抓取进度进行统一管理。输入任务表、原始详情表、预处理结果表和进度表相互配合，既保证了数据结果的保存，也支持任务去重、断点续爬和后续查询分析。

在数据提取方面，课题基于预处理结果表，围绕岗位方向分布、城市分布、高薪岗位特征和技能关键词需求等问题编写 SQL 查询语句，将大量岗位明细数据转化为可分析的统计结果，为后续可视化展示和结论分析提供了数据支撑。

总体而言，本课题较好地完成了互联网 IT 类实习岗位数据从网页采集到结构化处理、从数据库存储到统计提取的全过程，重点体现了 Selenium 自动化爬取、MySQL 数据管理、Pandas 数据预处理和 SQL 数据提取等关键技术。后续可进一步保留职位描述字段，并结合文本分析方法，对岗位职责和技能要求进行更深入的语义分析。

8 参考文献

- [1] 国家数据局. 数字中国发展报告（2024 年）[R/OL]. 北京: 国家数据局, 2025[2026-04-26].
- [2] 人力资源社会保障部, 中共中央组织部, 中央网信办, 等. 加快数字人才培养支撑数字经济发展行动方案（2024—2026 年）[EB/OL]. 2024[2026-04-26].
- [3] 中国互联网络信息中心. 第 56 次中国互联网络发展状况统计报告[R/OL]. 北京: 中国互联网络信息中心, 2025[2026-04-26].
- [4] 丁文浩, 朱齐亮. 基于 Python 的招聘数据爬取与分析[J]. 电脑编程技巧与维护, 2022(01): 43-45.
- [5] 项博良, 唐淳淳, 钱前, 等. 基于网络爬虫的就业数据分析[J]. 智能计算机与应用, 2020, 10(1): 223-227.
- [6] 刘新雨, 师云秋, 陈俊, 等. 基于大数据的智能招聘系统设计与实现[J]. 科技理论与实践, 2025, 6(4).
- [7] 中国信息通信研究院. 中国数字经济发展研究报告（2024 年）[R]. 北京: 中国信息通信研究院, 2024.
- [8] 贾柠瑜. 基于 Python 爬虫的岗位数据分析——以拉勾网为例[J]. 信息技术与信息化, 2019(4): 64-66.
- [9] 曾诚. 基于 Python 的网络爬虫及数据可视化和预测分析[J]. 信息与电脑(理论版), 2020(9): 167-169.
- [10] 陈清. 基于 Python 的网站爬虫应用研究[J]. 通讯世界, 2020, 27(1): 202-203.
- [11] 刘鑫. 网络爬虫在信息检索中的研究与应用[J]. 数字技术与应用, 2017(5): 95-97.
- [12] World Wide Web Consortium. WebDriver[EB/OL]. 2017[2026-04-26].
- [13] McKinney W. Data Structures for Statistical Computing in Python[C]//Proceedings of the 9th Python in Science Conference. Austin: SciPy, 2010: 56-61.
- [14] Hunter J D. Matplotlib: A 2D Graphics Environment[J]. Computing in Science & Engineering, 2007, 9(3): 90-95.
- [15] Waskom M L. seaborn: statistical data visualization[J]. Journal of Open Source Software, 2021, 6(60): 3021.
- [16] 黄岷昊, 丁浪, 张雪莲. 基于 Python 的网络爬虫及文本可视化[J]. 电脑编程技巧与维护, 2020(7): 24-25.